CS515 Deep learning: optimization and backpropagation

Emre Ozfatura[1]

[1] Intelligence System Lab,
Sabanci University, Turkey

23 February 2026

Brief summary

- We have identified 3 main aspects in deep learning:
- Define a model structure with learnable parameters, e.g., multi-layer perceptron
- Define the task-specific loss term , e.g., cross-entropy loss for classification.
- Update the parameters towards minimizing the loss terms: Optimizers

Risk minimization

Let $(x, y) \sim \mathcal{D}$ denote input-label pairs drawn from an unknown data distribution $\mathcal{D}$. Let $f_\theta : \mathcal{X} \to \mathcal{Y}$ be a deep neural network parameterized by $\theta \in \Theta$, and let $\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_+$ be a loss function.

- The *expected risk* of the model is defined as

$$\mathcal{R}(\boldsymbol{\theta}) = \mathbb{E}_{(x,y)\sim\mathcal{D}} \left[\mathcal{L}\big(f_{\boldsymbol{\theta}}(x), y\big) \right]. \tag{1}$$

- The goal of learning is to find parameters that minimize the expected risk:

$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}\in\Theta} \mathcal{R}(\boldsymbol{\theta}). \tag{2}$$

Empirical risk minimization

- Since the data distribution $\mathcal{D}$ is unknown, the expected risk cannot be computed.
- Given a training dataset $\mathcal{S} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ (i.i.d. samples from $\mathcal{D}$), the expected risk is approximated by *empirical risk*

$$\hat{\mathcal{R}}_n(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \mathcal{L}\big(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i\big). \tag{3}$$

- Training a deep neural network thus corresponds to solving the empirical risk minimization (ERM) problem:

$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}\in\Theta} \hat{\mathcal{R}}_n(\boldsymbol{\theta}). \tag{4}$$

- In practice, regularized empirical risk minimization objective is used:

$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}\in\Theta} \left[\frac{1}{n}\sum_{i=1}^{n} \mathcal{L}\big(f_{\boldsymbol{\theta}}(x_i), y_i\big) + \lambda\,\Omega(\boldsymbol{\theta})\right], \tag{5}$$

where $\Omega(\boldsymbol{\theta})$ is a regularization term and $\lambda > 0$ controls its strength.

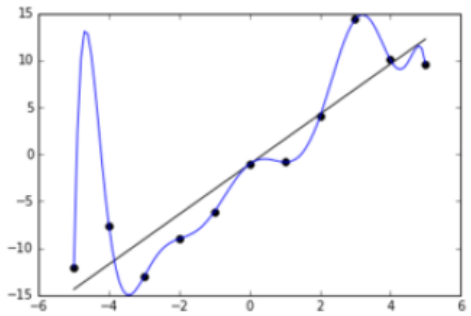
Overfitting Vs Underfitting with ERM

Empirical risk minimization

Empirical risk $= \frac{1}{n}\sum_{i=1}^{n}\mathcal{L}(x_i, y_i, \theta) \approx E_{x\sim p(x), y\sim p(y|x)}[\mathcal{L}(x, y, \theta)]$

Supervised learning is (usually) *empirical* risk minimization

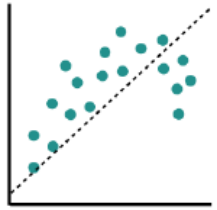
Is this the same as *true* risk minimization?



Overfitting: when the empirical risk is low, but the true risk is high

can happen if the dataset is too small

can happen if the model is too powerful (has too many parameters/capacity)



Underfitting: when the empirical risk is high, and the true risk is high

can happen if the model is too weak (has too few parameters/capacity)

can happen if your optimizer is not configured well (e.g., wrong learning rate)

Overfitting Vs Underfitting with ERM

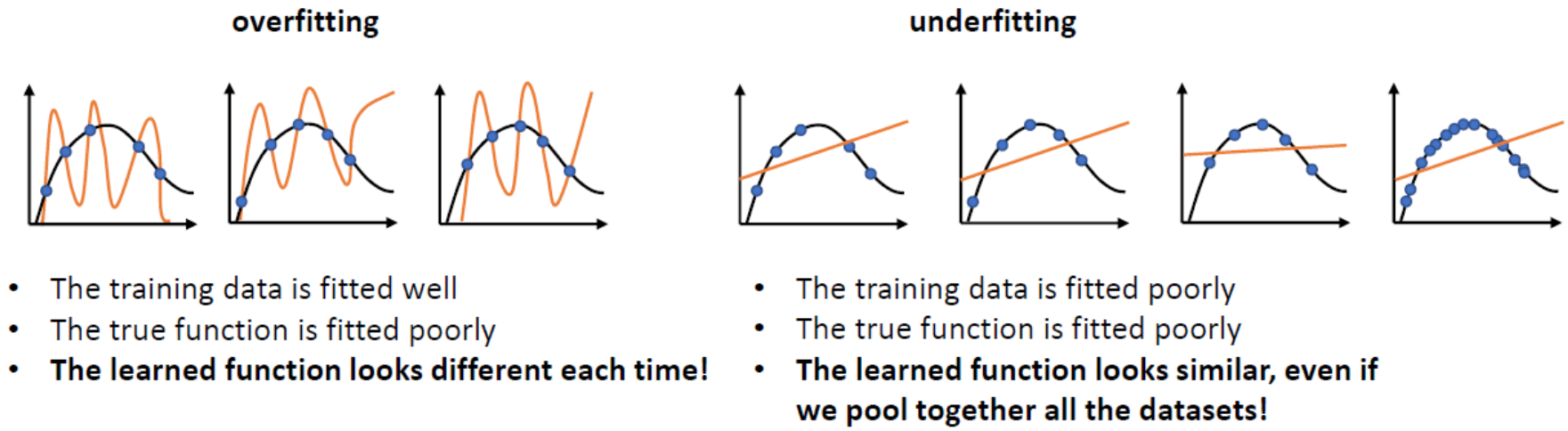


Figure 1: Illustration of overfitting and underfitting cases (Images from Berkeley CS 182; Sergey levine)

Bias-Variance tradeoff

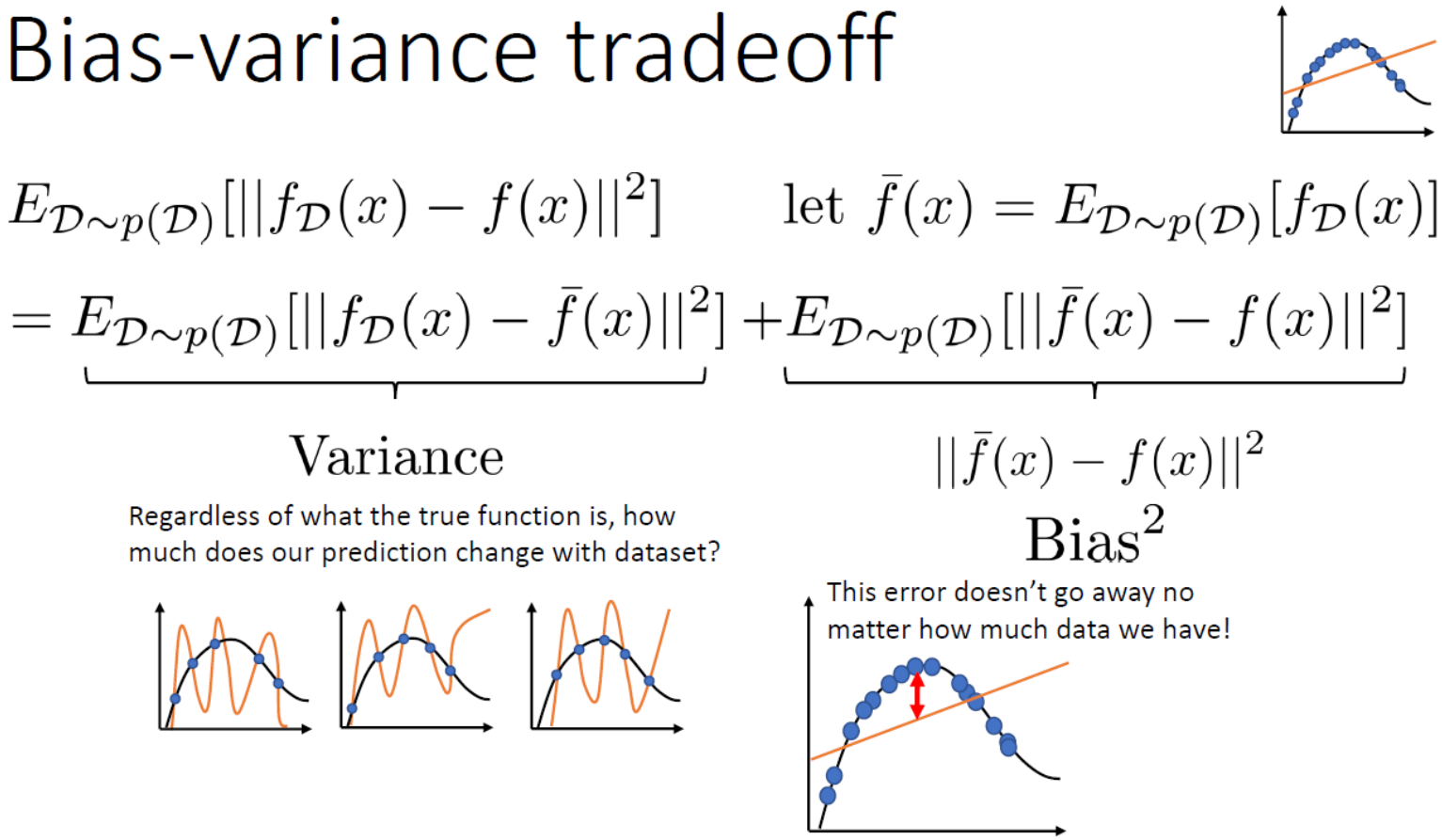


Figure 2: Illustration of Bias-Variance tradeoff (Images from Berkeley CS 182; Sergey levine)

Regularization from a MAP Perspective

- From a Bayesian viewpoint, the parameters $\boldsymbol{\theta}$ of a deep neural network are treated as random variables.
- Given a training dataset $\mathcal{S} = \{(x_i, y_i)\}_{i=1}^n$ and a likelihood model $p(y \mid x, \theta)$, Bayesian inference defines the posterior distribution over parameters via Bayes' rule:

$$p(\boldsymbol{\theta} \mid \mathcal{S}) \propto p(\mathcal{S} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta}), \tag{6}$$

  where $p(\theta)$ denotes a prior distribution encoding prior beliefs about the model parameters.
- Maximum a posteriori (MAP) estimation seeks the parameter value that maximizes the posterior probability:

$$\theta_{\mathrm{MAP}} = \arg\max_{\boldsymbol{\theta}} \; p(\boldsymbol{\theta} \mid \mathcal{S}). \tag{7}$$

- Taking the logarithm and using the monotonicity of the log function yields

$$\theta_{\mathrm{MAP}} = \arg\max_{\boldsymbol{\theta}} \left[\log p(\mathcal{S} \mid \boldsymbol{\theta}) + \log p(\boldsymbol{\theta})\right]. \tag{8}$$

Regularization from a MAP Perspective

- Assuming that the samples in $\mathcal{S}$ are independent and identically distributed, the likelihood factorizes as

$$p(\mathcal{S} \mid \boldsymbol{\theta}) = \prod_{i=1}^{n} p(y_i \mid x_i, \boldsymbol{\theta}). \tag{9}$$

- Defining the loss function as the negative log-likelihood,

$$\ell\big(f_{\boldsymbol{\theta}}(x_i), y_i\big) = -\log p(y_i \mid x_i, \boldsymbol{\theta}), \tag{10}$$

- MAP estimation can be equivalently written as the minimization problem

$$\theta_{\mathrm{MAP}} = \arg\min_{\boldsymbol{\theta}} \left[\sum_{i=1}^{n} \ell\big(f_{\boldsymbol{\theta}}(x_i), y_i\big) - \log p(\boldsymbol{\theta}) \right]. \tag{11}$$

Regularization from a MAP Perspective

- This objective coincides with regularized empirical risk minimization, where the regularization term arises naturally from the prior distribution, i.e.,

$$\Omega(\theta) = -\log p(\theta), \tag{12}$$

- Now notice that ℓ_2 regularization,

$$\Omega(\theta) = \|\theta\|_2^2, \tag{13}$$

corresponds to minimizing NLL of prior distribution, $-\log p(\boldsymbol{\theta})$, under the assumption

$$\boldsymbol{\theta} \sim \mathcal{N}(0, \sigma^2 I). \tag{14}$$

- Similarly, ℓ_1 regularization,

$$\Omega(\theta) = \|\theta\|_1, \tag{15}$$

corresponds to a Laplace prior, which encourages sparsity in the learned parameters.

Regularization with dropout

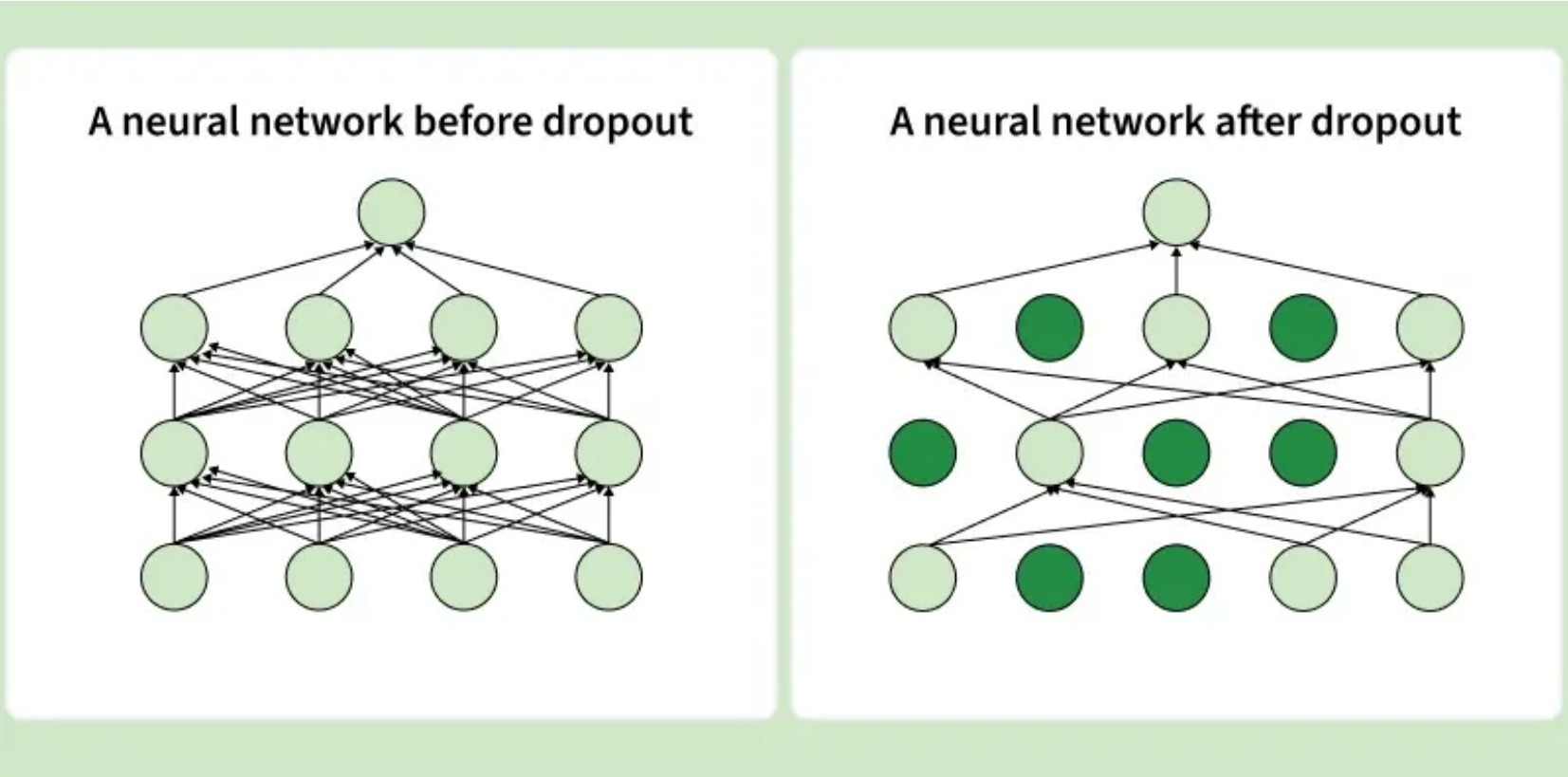
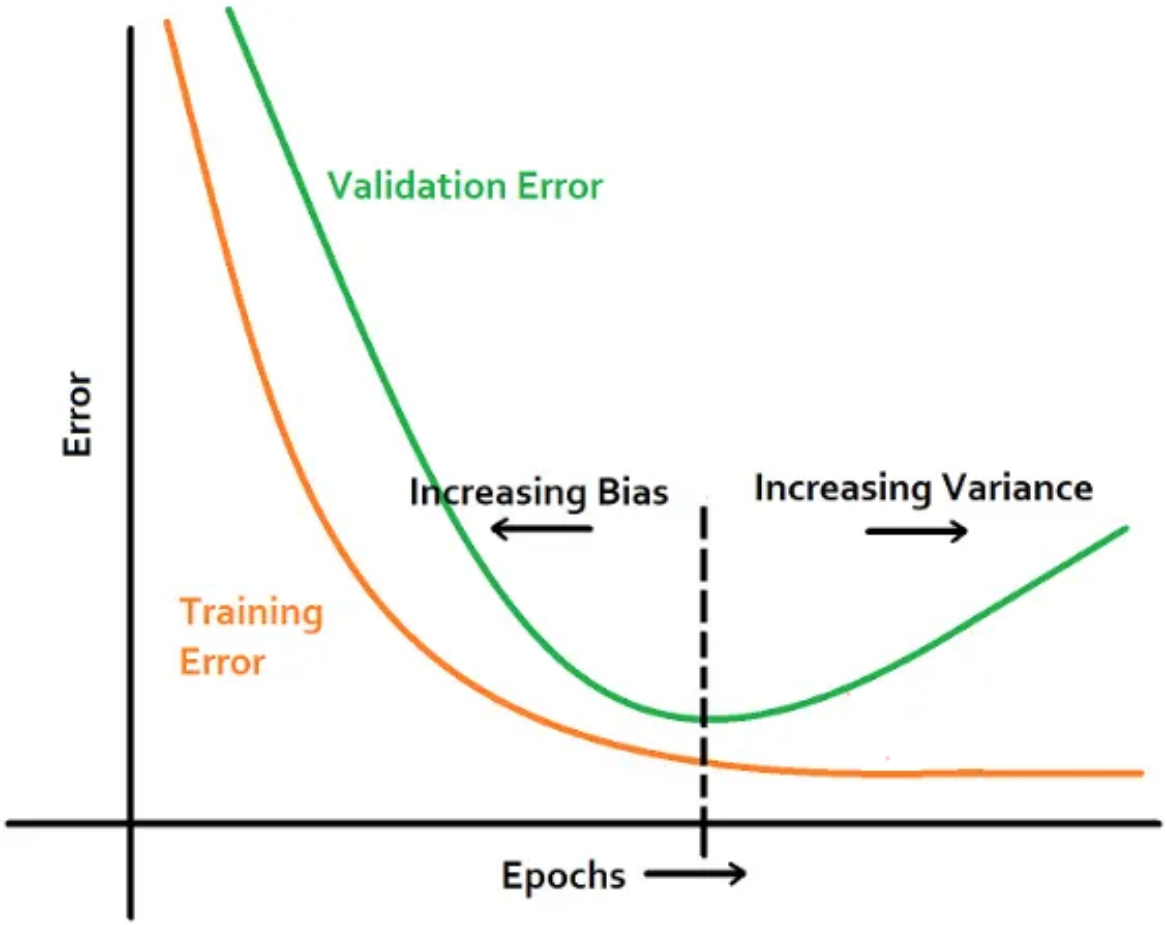


Figure 3: Illustration of dropout during test and training phases.

Early stopping to prevent overfitting



- Want to reduce the bias without havinghigh variance.
- Later we will visit how to tune hyperparametters and use validation loss accordingly.

Loss landscape

The loss "landscape"

$$\theta^\star \leftarrow \arg\min_\theta \underbrace{-\sum_i \log p_\theta(y_i|x_i)}_{\mathcal{L}(\theta)}$$

let's say θ is 2D

An algorithm:

1. Find a *direction* v where $\mathcal{L}(\theta)$ decreases
2. $\theta \leftarrow \theta + \alpha v$

some small constant called "learning rate" or "step size"

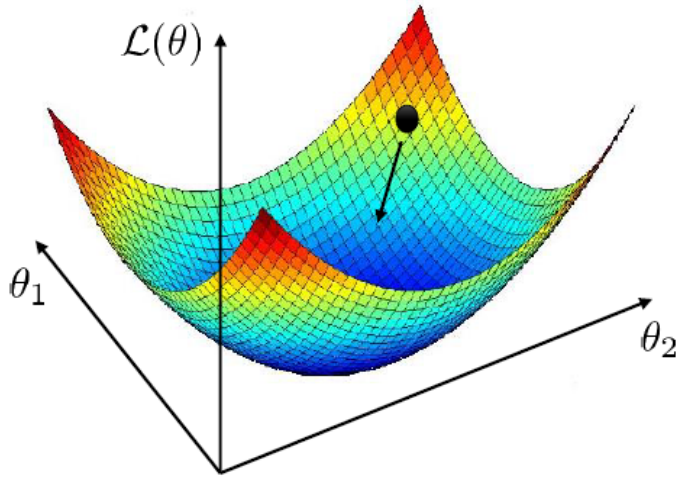


Figure 4: Illustration of loss landscape (Images from Berkeley CS 182; Sergey levine)

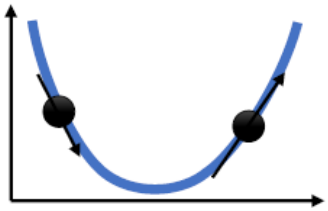
Loss landscape and gradient descent

Gradient descent

An algorithm:

1. Find a *direction* v where $\mathcal{L}(\theta)$ decreases
2. $\theta \leftarrow \theta + \alpha v$

Which way does $\mathcal{L}(\theta)$ decrease?



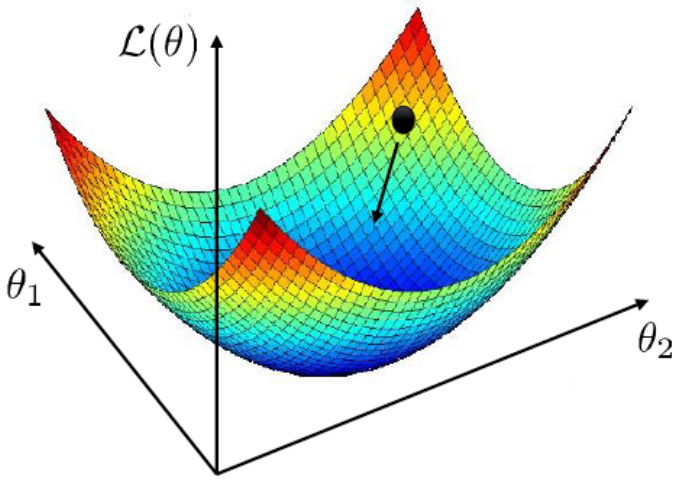
negative slope = go to the right

positive slope = go to the left

in general:

for each dimension, go in the direction opposite the slope **along that dimension**

$$v_1 = -\frac{d\mathcal{L}(\theta)}{d\theta_1} \quad v_2 = -\frac{d\mathcal{L}(\theta)}{d\theta_2} \quad \text{etc.}$$



gradient:

$$\nabla_\theta \mathcal{L}(\theta) = \begin{pmatrix} \frac{d\mathcal{L}(\theta)}{d\theta_1} \\ \frac{d\mathcal{L}(\theta)}{d\theta_2} \\ \vdots \\ \frac{d\mathcal{L}(\theta)}{d\theta_n} \end{pmatrix}$$

Figure 5: Illustration of gradient descent framework (Images from Berkeley CS 182; Sergey levine)

Gradient descent framework

- Given an objective function $f : \mathbb{R}^d \to \mathbb{R}$, gradient descent (GD) iteratively updates the parameters as

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \gamma \nabla f(\mathbf{x}_t), \tag{16}$$

  where $\gamma > 0$ is the step size (learning rate).
- BE CAREFUL !! we use notation $\mathbf{x}_t$, to be aligned with convex opt. literature, but in our setup it is actually $\boldsymbol{\theta}_t$

Definition (Convex Function)

Let $f : \mathbb{R}^d \to \mathbb{R} \cup \{+\infty\}$. We say that f is *convex* if for all $x, y \in \mathbb{R}^d$ and for all $\lambda \in (0, 1)$,

$$f\big(\lambda x + (1-\lambda)y\big) \leq \lambda f(x) + (1-\lambda) f(y). \tag{17}$$

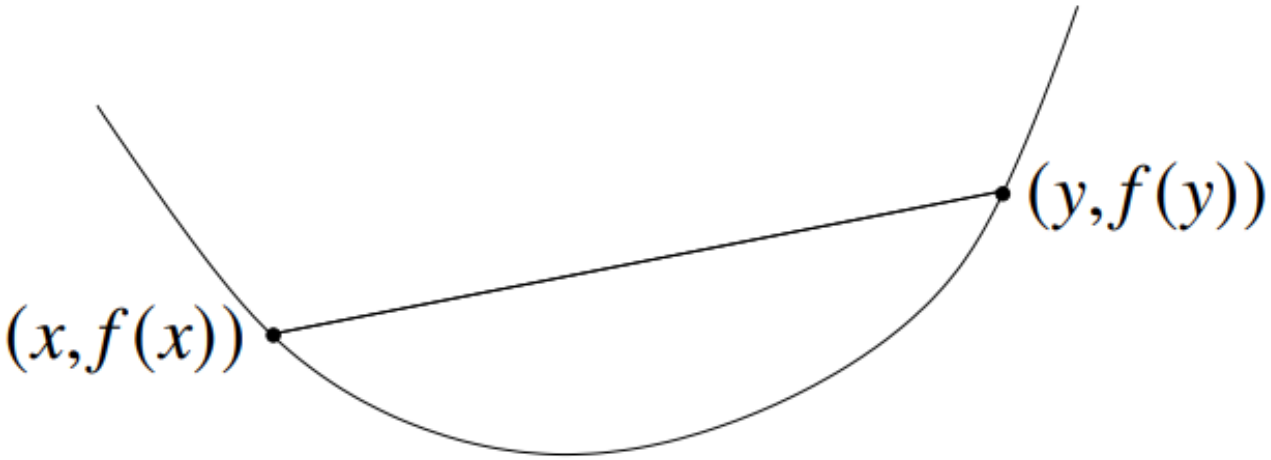


Figure 6: Illustration of convex function

First order condition

Lemma (Lower-bound approximation)

Let $f : \mathbb{R}^d \to \mathbb{R}$ *be convex and differentiable. Then, for all* $x, y \in \mathbb{R}^d$,

$$f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle. \tag{18}$$

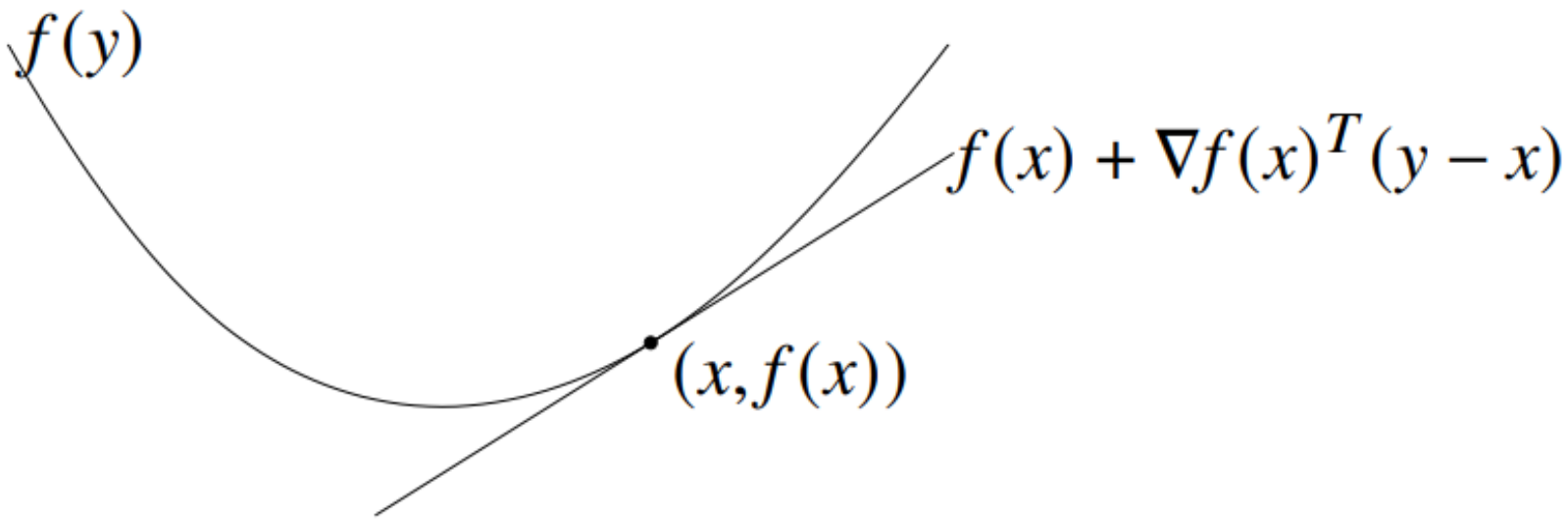


Figure 7: Illustration of first order condition

Gradient Descent Theorem for Convex and Smooth Functions

Consider a differentiable function f and assume that f is convex and L-smooth, for some $L > 0$. Let $(x^t)_{t \in \mathbb{N}}$ be the sequence of iterates generated by the (GD) algorithm, with a step-size satisfying $0 < \gamma < \frac{1}{L}$. Then, for all $\mathbf{x}^\star \in \arg\min f$, for all $t \in \mathbb{N}$ we have

$$f(\mathbf{x}^t) - \inf f \;\leq\; \frac{\|\mathbf{x}^0 - \mathbf{x}^\star\|^2}{2\gamma t}. \tag{19}$$

Definition (L-smooth function)

Let $f : \mathbb{R}^d \to \mathbb{R}$ and let $L > 0$. We say that f is *L-smooth* if it is differentiable and its gradient $\nabla f : \mathbb{R}^d \to \mathbb{R}^d$ is L-Lipschitz continuous, i.e., for all $x, y \in \mathbb{R}^d$,

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|. \tag{20}$$

Lemma (Smoothness upper bound)

Let $f : \mathbb{R}^d \to \mathbb{R}$ be L-smooth. Then, for all $x, y \in \mathbb{R}^d$,

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2}\|y - x\|^2. \tag{21}$$

Proof of GD Theorem

- **Monotonicity of the function values:** using GD update, $\mathbf{x}_{t+1} = \mathbf{x}_t - \gamma \nabla f(\mathbf{x}_t)$, and smoothnss upper-bound eqn. (21) we have

$$f(\mathbf{x}_{t+1}) - f(\mathbf{x}_t) \leq -\gamma \left(1 - \frac{\gamma L}{2}\right) \|\nabla f(\mathbf{x}_t)\|^2. \tag{22}$$

- Since, we assume $\gamma L \leq 1$, it follows that

$$f(\mathbf{x}_{t+1}) \leq f(\mathbf{x}_t), \tag{23}$$

and thus $(f(\mathbf{x}_t))_{t \in \mathbb{N}}$ is a non-increasing sequence.

- How distance to $x^\star$ evolves from $\mathbf{x}_{t+1}$ to $\mathbf{x}_{t+1}$:

$$\frac{1}{2\gamma}\|\mathbf{x}_{t+1}-\mathbf{x}^\star\|^2-\frac{1}{2\gamma}\|\mathbf{x}_t-\mathbf{x}^\star\|^2=$$
$$-\frac{1}{2\gamma}\|\mathbf{x}_{t+1}-\mathbf{x}_t\|^2-\langle\nabla f(x_t),x_{t+1}-\mathbf{x}^\star\rangle=$$
$$-\frac{1}{2\gamma}\|\mathbf{x}_{t+1}-\mathbf{x}_t\|^2-\langle\nabla f(\mathbf{x}_t),x_{t+1}-\mathbf{x}^\star-\mathbf{x}_t+\mathbf{x}_t\rangle= \tag{24}$$
$$-\frac{1}{2\gamma}\|x_{t+1}-x_t\|^2-\langle\nabla f(\mathbf{x}_t),\mathbf{x}_{t+1}-\mathbf{x}_t\rangle+\langle\nabla f(\mathbf{x}_t),\mathbf{x}^\star-\mathbf{x}_t\rangle. \tag{25}$$

- Hint: above we utilize the following identity

$$\|\mathbf{a}\|^2-\|\mathbf{b}\|^2=\|\mathbf{a}-\mathbf{b}\|^2+2\langle\mathbf{b},\mathbf{a}-\mathbf{b}\rangle \tag{26}$$

- Bound first inner product using first-order condition for convex functions eqn. (18),

$$\langle\nabla f(\mathbf{x}_t),\mathbf{x}^\star-\mathbf{x}_t\rangle\leq f(\mathbf{x}^\star)-f(\mathbf{x}_t)=f_{inf}-f(\mathbf{x}_t). \tag{27}$$

- Bound the second inner product using smoothness upper bound

$$-\langle \nabla f(\mathbf{x}_t), \mathbf{x}_{t+1} - \mathbf{x}_t \rangle \leq \frac{L}{2}\|\mathbf{x}_{t+1} - \mathbf{x}_t\|^2 + f(\mathbf{x}_t) - f(\mathbf{x}_{t+1}). \quad (28)$$

- Substituting (27) and (28) into (25), we obtain

$$\begin{aligned} &\frac{1}{2\gamma}\|\mathbf{x}_{t+1} - \mathbf{x}^\star\|^2 - \frac{1}{2\gamma}\|\mathbf{x}_t - \mathbf{x}^\star\|^2 && (29) \\ &\leq \frac{\gamma L - 1}{2\gamma}\|\mathbf{x}_{t+1} - \mathbf{x}_t\|^2 - \big(f(\mathbf{x}_{t+1}) - \inf f\big) \\ &\leq -\big(f(\mathbf{x}_{t+1}) - \inf f\big), && (30) \end{aligned}$$

where the last inequality follows from $\gamma L \leq 1$.

- **Lyapunov energy and convergence rate:** Define the Lyapunov energy

$$E_t := \frac{1}{2\gamma}\|\mathbf{x}_t - \mathbf{x}^\star\|^2 + t\big(f(\mathbf{x}_t) - \inf f\big), \qquad t \in \mathbb{N}. \tag{31}$$

- Compute sequential difference:

$$\begin{aligned} E_{t+1} - E_t = (t+1)\big(f(\mathbf{x}_{t+1}) - \inf f\big) - t\big(f(\mathbf{x}_t) - \inf f\big) + \\ \frac{1}{2\gamma}\|\mathbf{x}_{t+1} - \mathbf{x}^\star\|^2 - \frac{1}{2\gamma}\|\mathbf{x}_t - \mathbf{x}^\star\|^2 \\ = f(x_{t+1}) - \inf f + t\big(f(\mathbf{x}_{t+1}) - f(\mathbf{x}_t)\big) + \\ \frac{1}{2\gamma}\|\mathbf{x}_{t+1} - \mathbf{x}^\star\|^2 - \frac{1}{2\gamma}\|\mathbf{x}_t - \mathbf{x}^\star\|^2. \end{aligned} \tag{32}$$

- Using (23) and (30) in (32), we obtain

$$\begin{aligned} E_{t+1} - E_t &\leq f(x_{t+1}) - \inf f + \frac{1}{2\gamma}\|x_{t+1} - x^\star\|^2 - \frac{1}{2\gamma}\|x_t - x^\star\|^2 \\ &\leq f(x_{t+1}) - \inf f - \big(f(x_{t+1}) - \inf f\big) \\ &= 0. \end{aligned} \tag{33}$$

- Hence $(E_t)_{t\in\mathbb{N}}$ is non-increasing.
- Since $E_t \geq 0$, we conclude

$$t\big(f(x_t) - \inf f\big) \leq E_t \leq E_0 = \frac{1}{2\gamma}\|x_0 - x^\star\|^2,$$

which yields

$$f(x_t) - \inf f \leq \frac{\|x_0 - x^\star\|^2}{2\gamma t}. \tag{34}$$

GD versus Stochastic GD

- Iteration under GD:
  - Compute gradient

$$\nabla \hat{\mathcal{R}}(\boldsymbol{\theta}^t) = \frac{1}{n} \sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}} \mathcal{L}\big(f_{\boldsymbol{\theta}^t}(x_i), y_i\big) \tag{35}$$

  - Update model $\boldsymbol{\theta}^{t+1} = \boldsymbol{\theta}^t - \gamma \nabla \hat{\mathcal{R}}(\boldsymbol{\theta}^t)$.
- Iteration under SGD:
  - Select a random index $i_t \in \{1, \ldots, n\}$ and compute

$$g_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}\big(f_{\boldsymbol{\theta}^t}(x_{i_t}), y_{i_t}\big). \tag{36}$$

  - Update model $\boldsymbol{\theta}^{t+1} = \boldsymbol{\theta}^t - \gamma g_t$.
- In general, we use mini-batch SGD, where we sample $\mathcal{B} \in \mathcal{D}$ to compute gradient for better accuracy lower computational cost.

The geography of a loss landscape

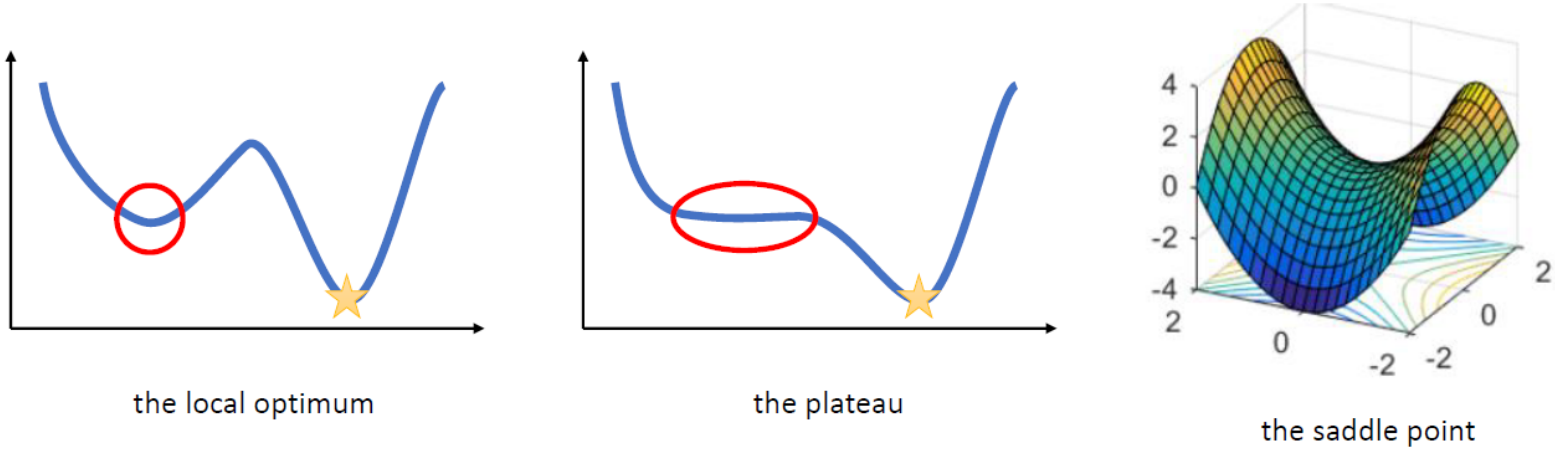


Figure 8: Illustration of convex function

Acceleration methods: Momentum SGD

- Select a random index $i_t \in \{1, \ldots, n\}$ and compute

$$g_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}\big(f_{\boldsymbol{\theta}^t}(x_{i_t}), y_{i_t}\big). \tag{37}$$

- Update momentum: $\mathbf{m}_t = \mathbf{m}_t \mu + g_t$

- Update model: $\boldsymbol{\theta}^{t+1} = \boldsymbol{\theta}^t - \gamma \mathbf{m}_t$.
- We reduce the variance and accelerate the convergence

Nesterov accelerated gradient

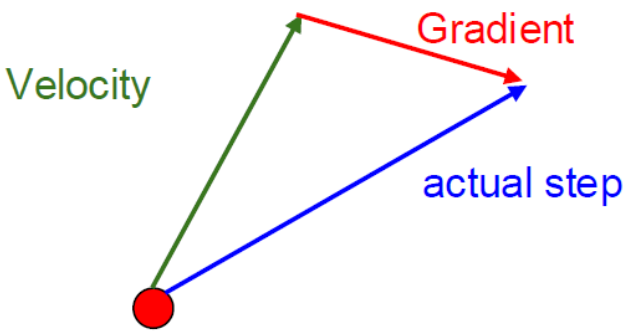


Figure 9: Illustration of convex function

- Take gradient at look ahead point

$$g_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}\big(f_{\boldsymbol{\theta}^{t+1/2}}(x_{i_t}), y_{i_t}\big). \tag{38}$$

  where, $\boldsymbol{\theta}^{t+1/2} = \boldsymbol{\theta} + \mu \mathbf{m}_t$
- Then the momentum term is updated as

$$\mathbf{m}_{t+1} = \mu \mathbf{m}_t - \alpha g_t \tag{39}$$

- Update the model $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + -\mathbf{m}_{t+1}$

Adam Optimizer: adaptive learning rate

- Adam maintains an exponential moving average of gradients:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) g_t, \tag{40}$$

  where $\beta_1 \in [0, 1)$ (typically $\beta_1 = 0.9$).
- Adam also tracks squared gradients:

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) g_t^2, \tag{41}$$

  where $\beta_2 \in [0, 1)$ (typically $\beta_2 = 0.999$).
- Bias Correction: since m_t and v_t are initialized at zero, they are biased toward zero. The bias-corrected estimates are:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \qquad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}.$$

- Parameter Update:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon}, \tag{42}$$

Decoupling weight decay

AdamW

```
class torch.optim.AdamW(params, lr=0.001, betas=(0.9, 0.999), eps=1e-08, weight_decay=0.01,
amsgrad=False, *, maximize=False, foreach=None, capturable=False, differentiable=False,
fused=None)
```
[source]

Implements AdamW algorithm, where weight decay does not accumulate in the momentum nor variance.

---

input : γ(lr), β_1, β_2(betas), θ_0(params), $f(\theta)$(objective), ϵ (epsilon)
λ(weight decay), *amsgrad*, *maximize*

initialize : $m_0 \leftarrow 0$ (first moment), $v_0 \leftarrow 0$ (second moment), $v_0^{max} \leftarrow 0$

---

for $t = 1$ **to** $\ldots$ **do**

  if *maximize* :

    $g_t \leftarrow -\nabla_\theta f_t(\theta_{t-1})$

  else

    $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$

  $\theta_t \leftarrow \theta_{t-1} - \gamma\lambda\theta_{t-1}$

  $m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1) g_t$

  $v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2) g_t^2$

  $\widehat{m_t} \leftarrow m_t / (1-\beta_1^t)$

  if *amsgrad*

    $v_t^{max} \leftarrow \max(v_{t-1}^{max}, v_t)$

    $\widehat{v_t} \leftarrow v_t^{max} / (1-\beta_2^t)$

  else

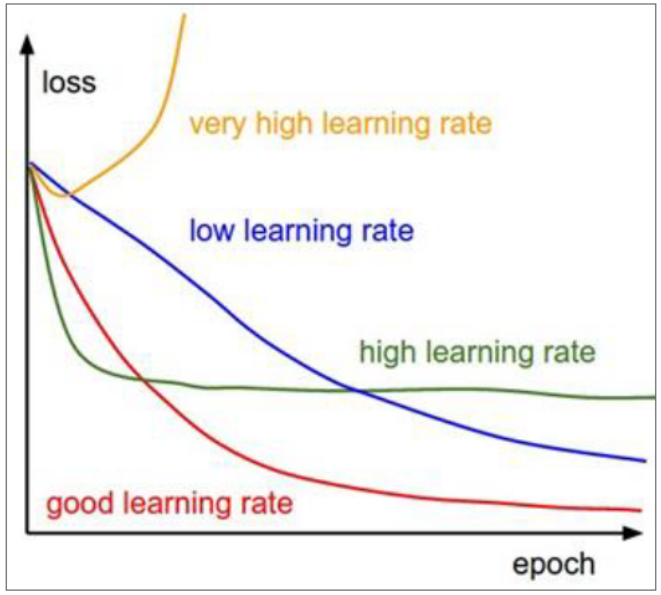
    $\widehat{v_t} \leftarrow v_t / (1-\beta_2^t)$

  $\theta_t \leftarrow \theta_t - \gamma \widehat{m_t} / (\sqrt{\widehat{v_t}} + \epsilon)$

---

return θ_t

SGD, SGD+Momentum, RMSProp, Adam, AdamW all have **learning rate** as a hyperparameter.

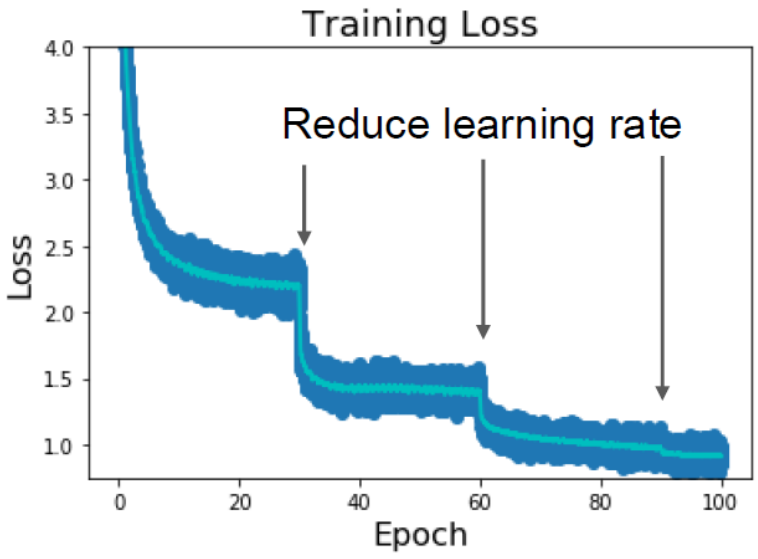


Q: Which one of these learning rates is best to use?

A: In reality, all of these could be good learning rates.

Figure 10: Learning rate decay

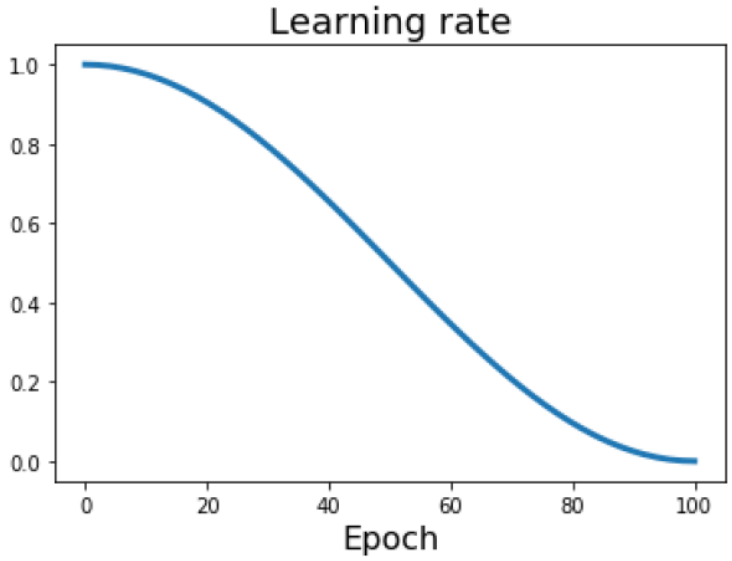
Learning rate decays over time



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Figure 11: Learning rate decay

Learning Rate Decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

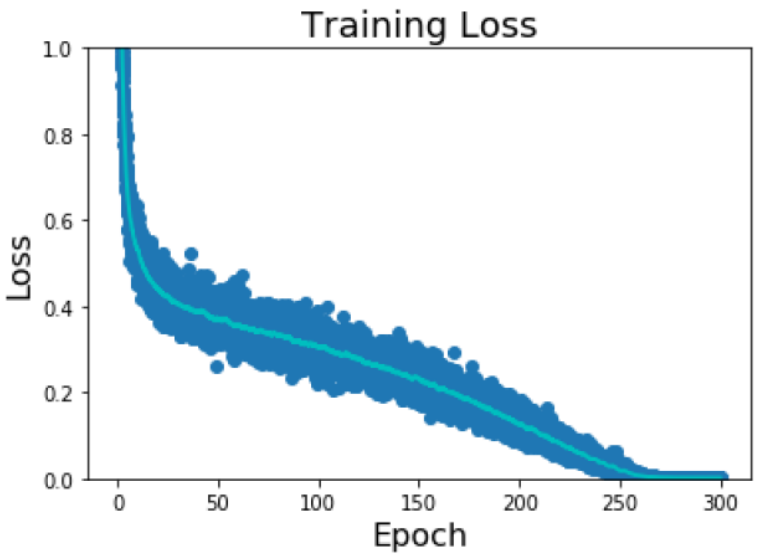
Cosine: $\alpha_t = \frac{1}{2}\alpha_0 \left(1 + \cos(t\pi/T)\right)$

α_0 : Initial learning rate
α_t : Learning rate at epoch t
T : Total number of epochs

Loshchilov and Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts", ICLR 2017
Radford et al, "Improving Language Understanding by Generative Pre-Training", 2018
Feichtenhofer et al, "SlowFast Networks for Video Recognition", arXiv 2018
Child at al, "Generating Long Sequences with Sparse Transformers", arXiv 2019

Figure 12: Learning rate decay

Learning Rate Decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

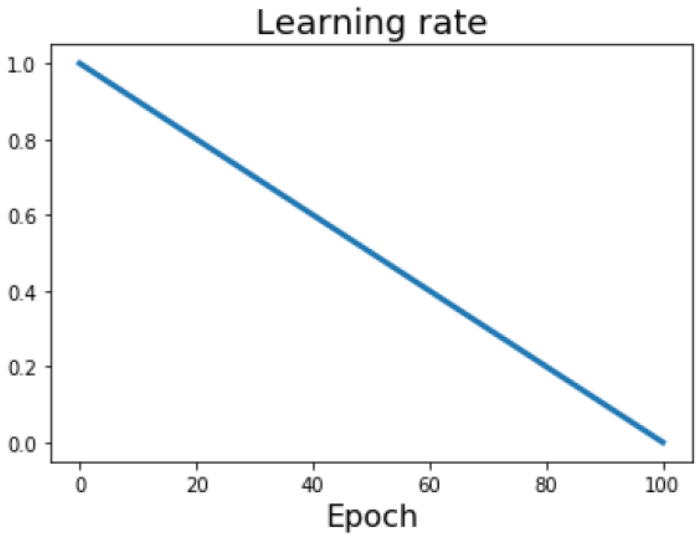
Cosine: $\alpha_t = \frac{1}{2}\alpha_0 \left(1 + \cos(t\pi/T)\right)$

α_0 : Initial learning rate
α_t : Learning rate at epoch t
T : Total number of epochs

Loshchilov and Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts", ICLR 2017
Radford et al, "Improving Language Understanding by Generative Pre-Training", 2018
Feichtenhofer et al, "SlowFast Networks for Video Recognition", arXiv 2018
Child at al, "Generating Long Sequences with Sparse Transformers", arXiv 2019

Figure 13: Learning rate decay

Learning Rate Decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2}\alpha_0 \left(1 + \cos(t\pi/T)\right)$

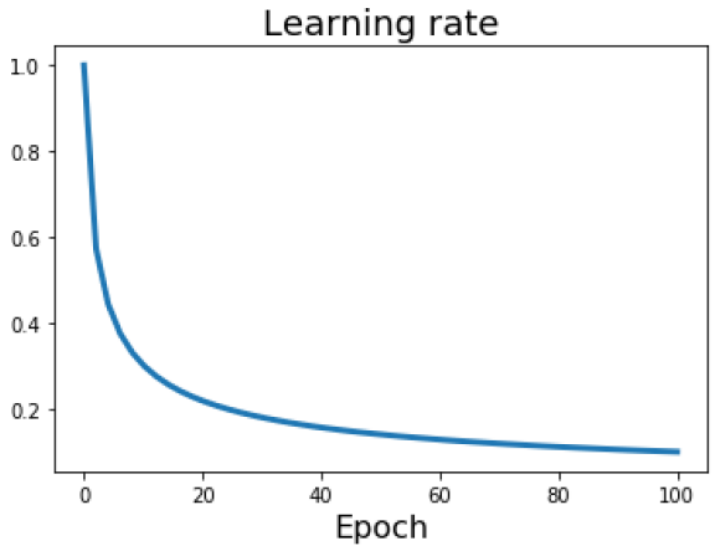
Linear: $\alpha_t = \alpha_0(1 - t/T)$

α_0 : Initial learning rate
α_t : Learning rate at epoch t
T : Total number of epochs

Devlin et al, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", 2018

Figure 14: Learning rate decay

Learning Rate Decay



Vaswani et al, "Attention is all you need", NIPS 2017

Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2}\alpha_0 \left(1 + \cos(t\pi/T)\right)$

Linear: $\alpha_t = \alpha_0(1 - t/T)$

Inverse sqrt: $\alpha_t = \alpha_0/\sqrt{t}$

α_0 : Initial learning rate
α_t : Learning rate at epoch t
T : Total number of epochs

Figure 15: Learning rate decay

```
>>> optimizer = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)
>>> scheduler = ReduceLROnPlateau(optimizer, "min")
>>> for epoch in range(10):
>>>     train(...)
>>>     val_loss = validate(...)
>>> # Note that step should be called after validate()
>>>     scheduler.step(val_loss)
```

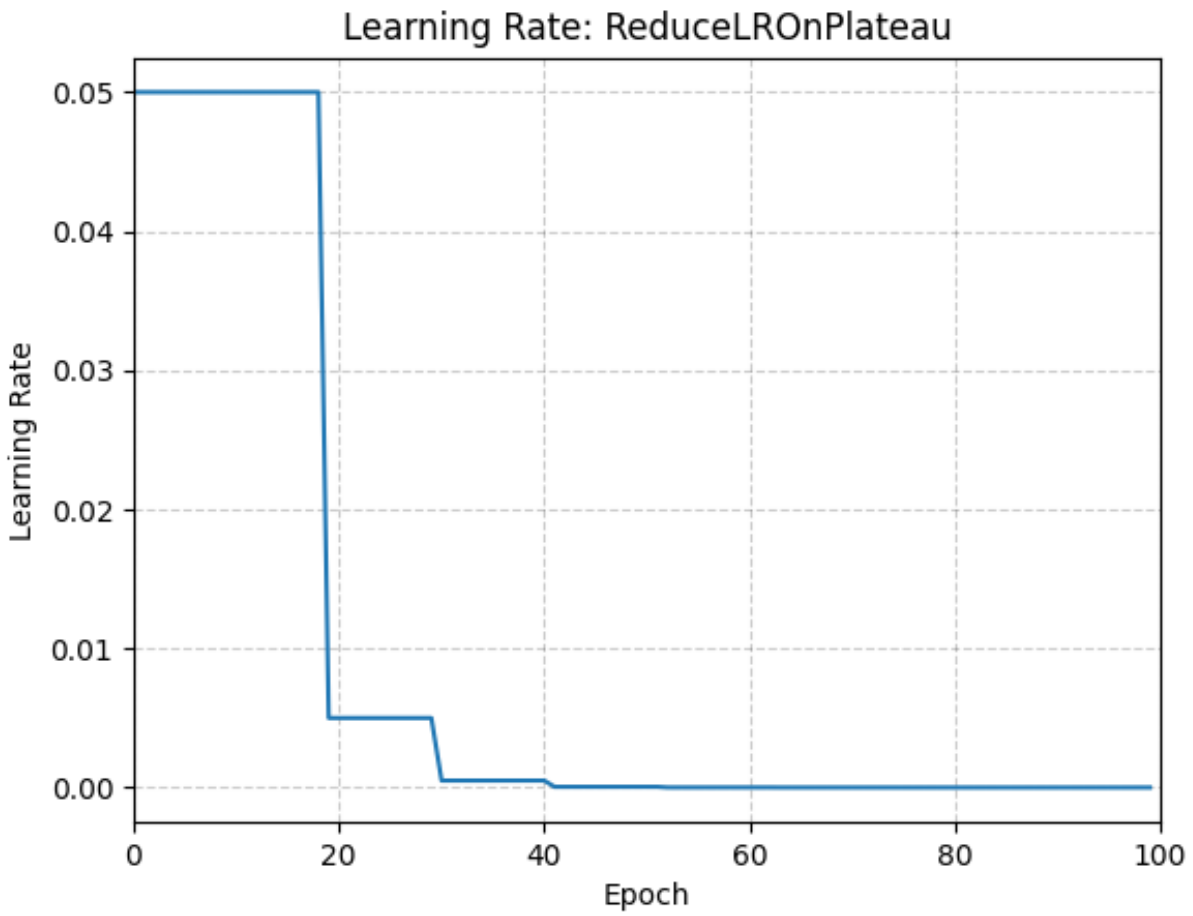
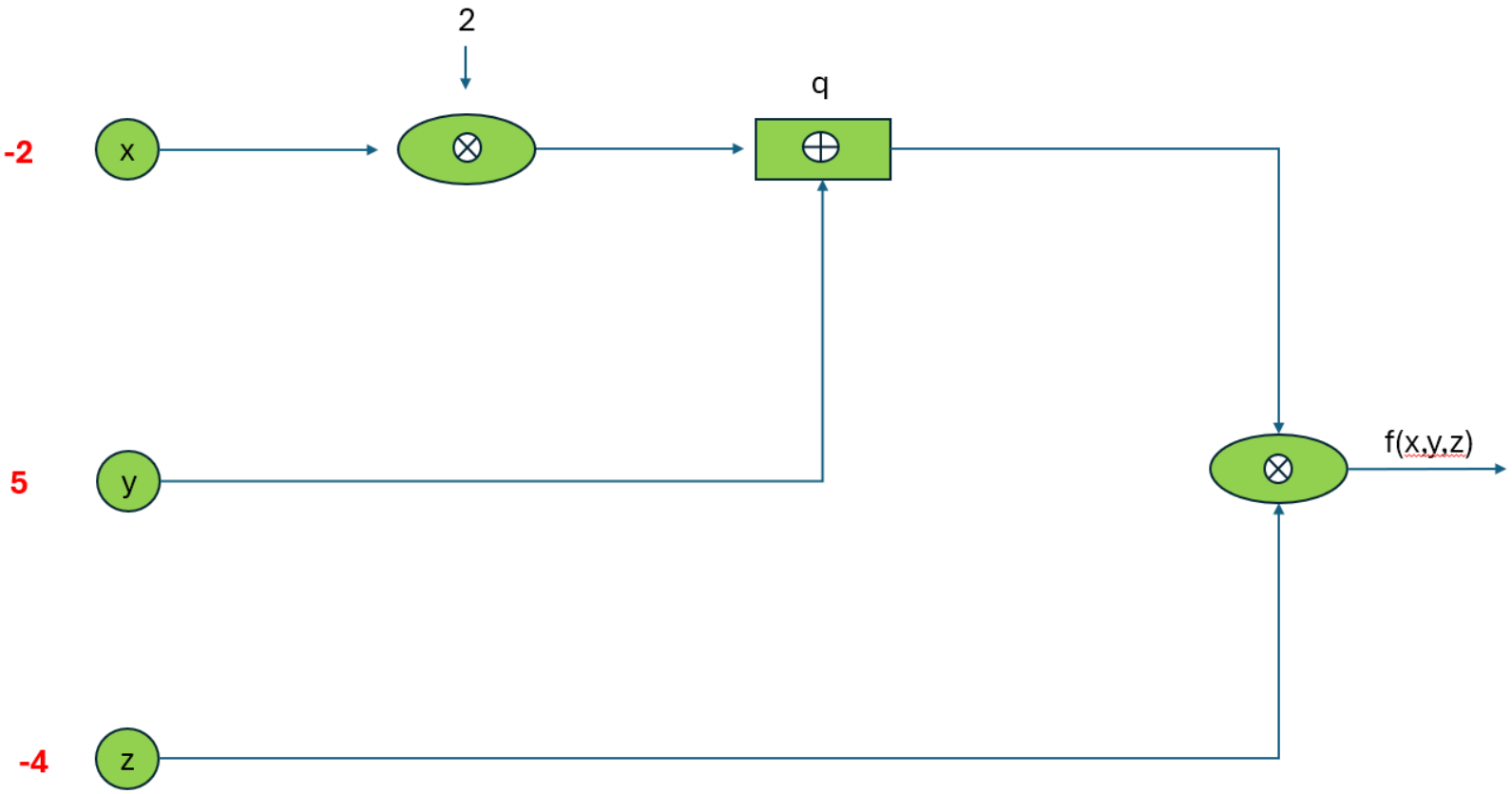


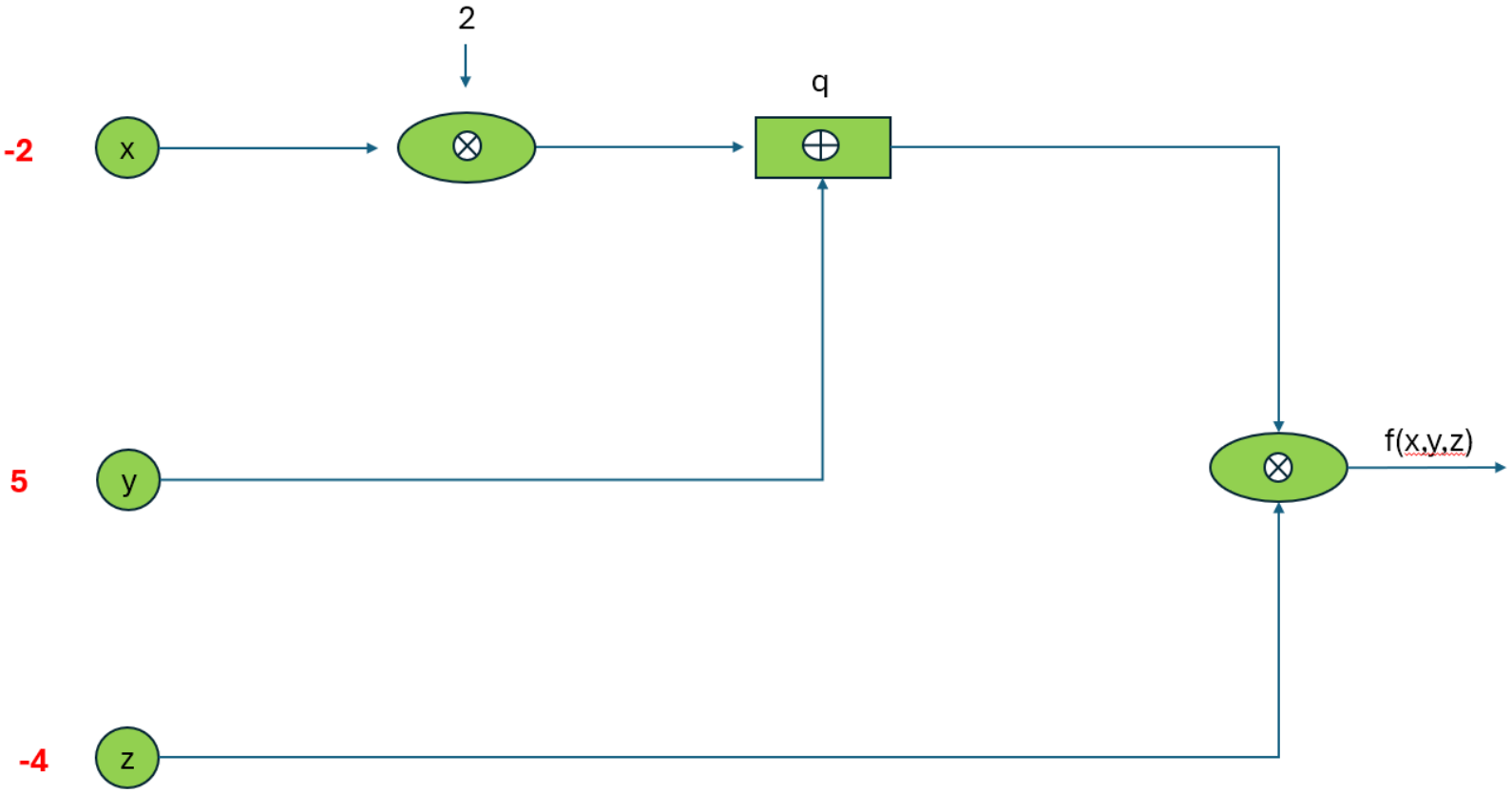
Figure 16: learning rate decay

Back Propagation



- $f(x, y, z) = (2x + y)z, q = 2x + y$
- Want to compute $\frac{\partial f}{\partial z}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial x}$

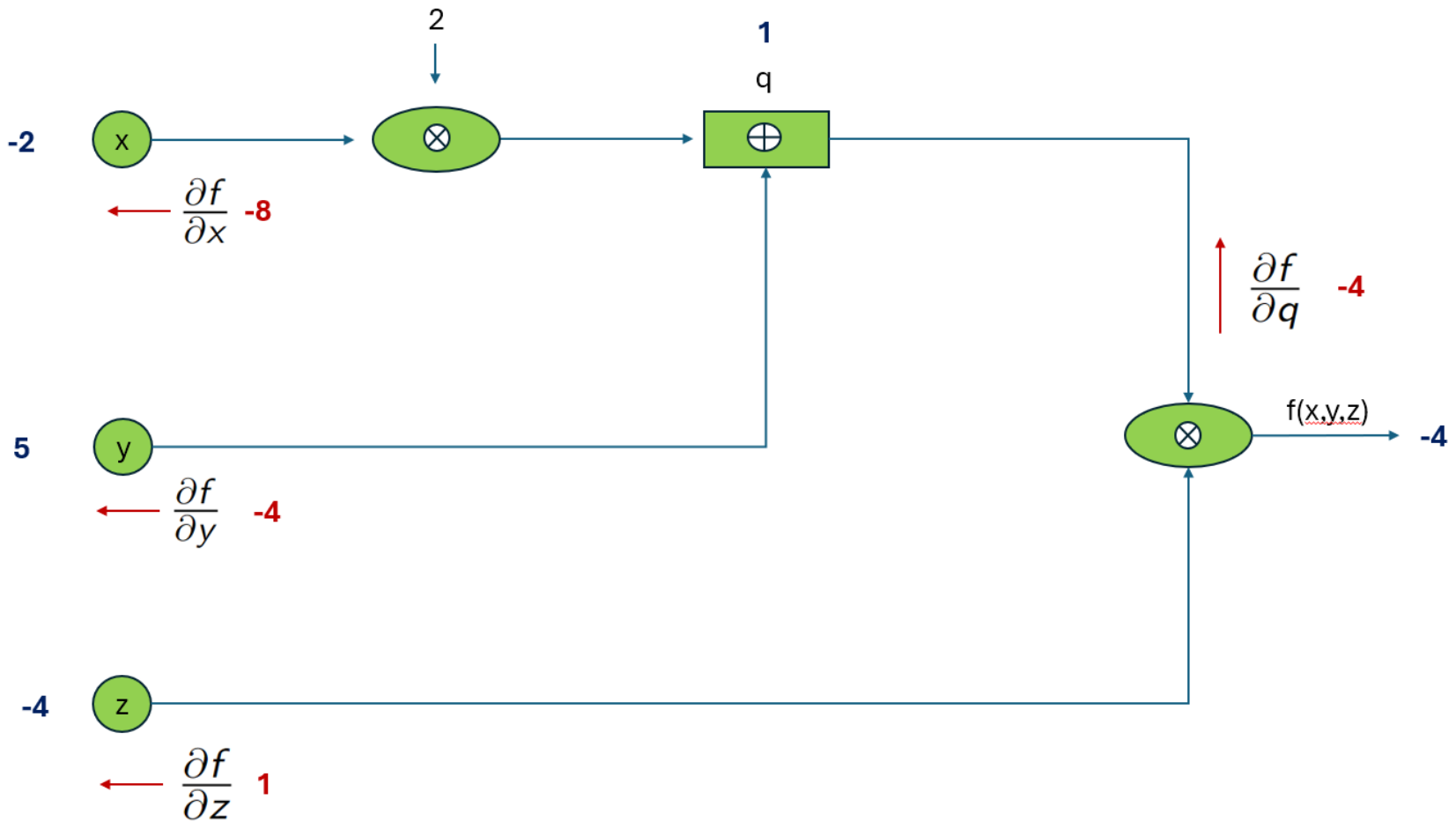
Back Propagation



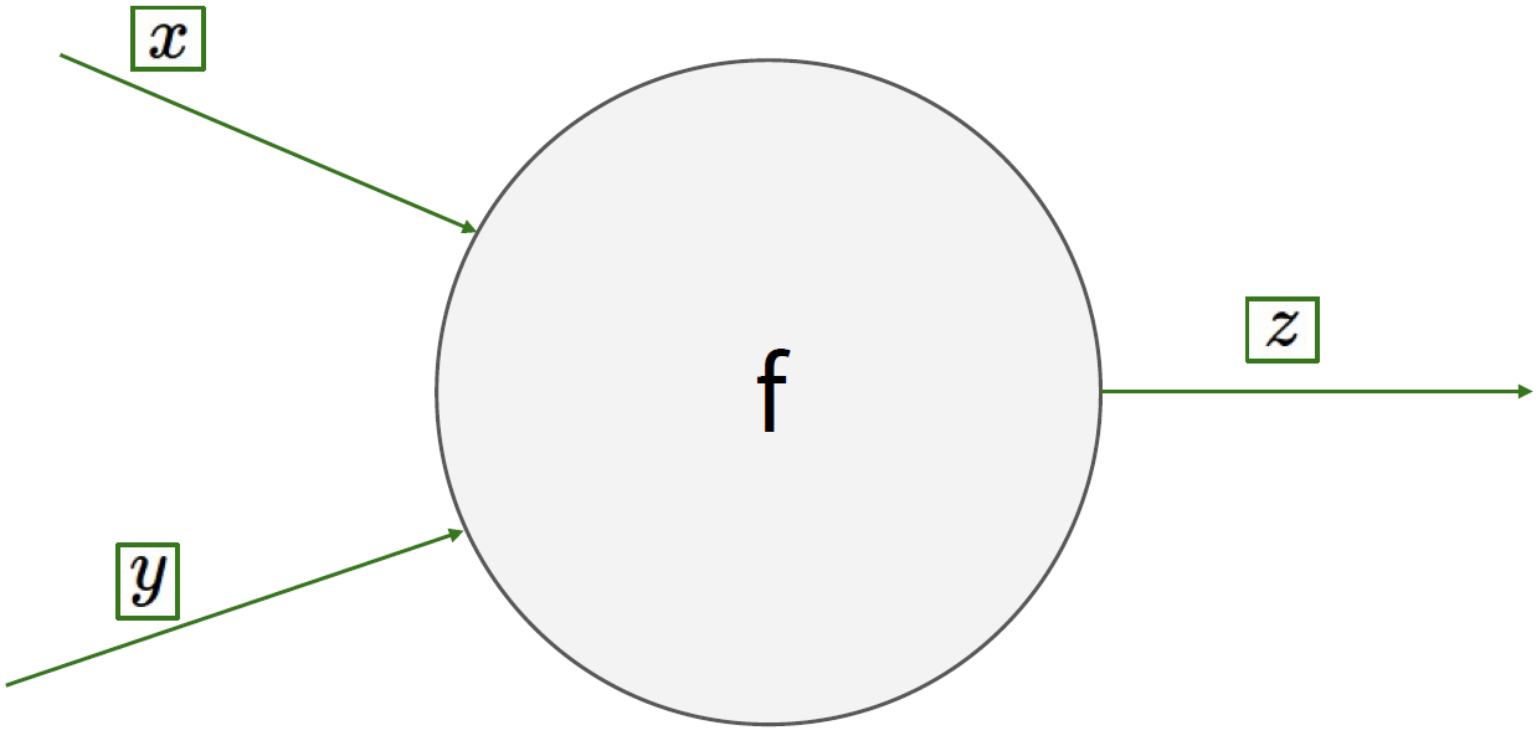
- $\frac{\partial q}{\partial x} = 2, \frac{\partial q}{\partial y} = 1$
- $\frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

Chain rule (local and upstream gradient): $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$

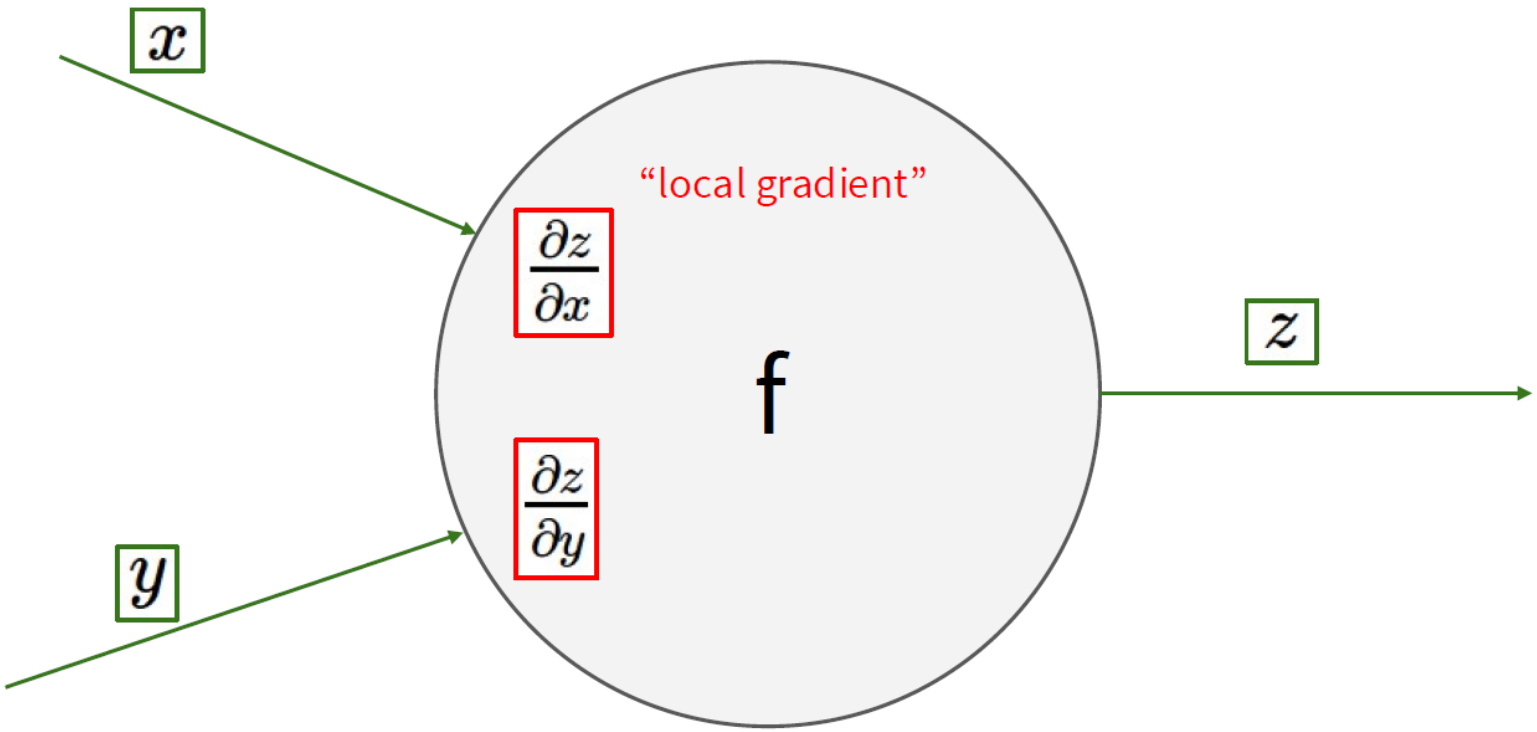
Back Propagation



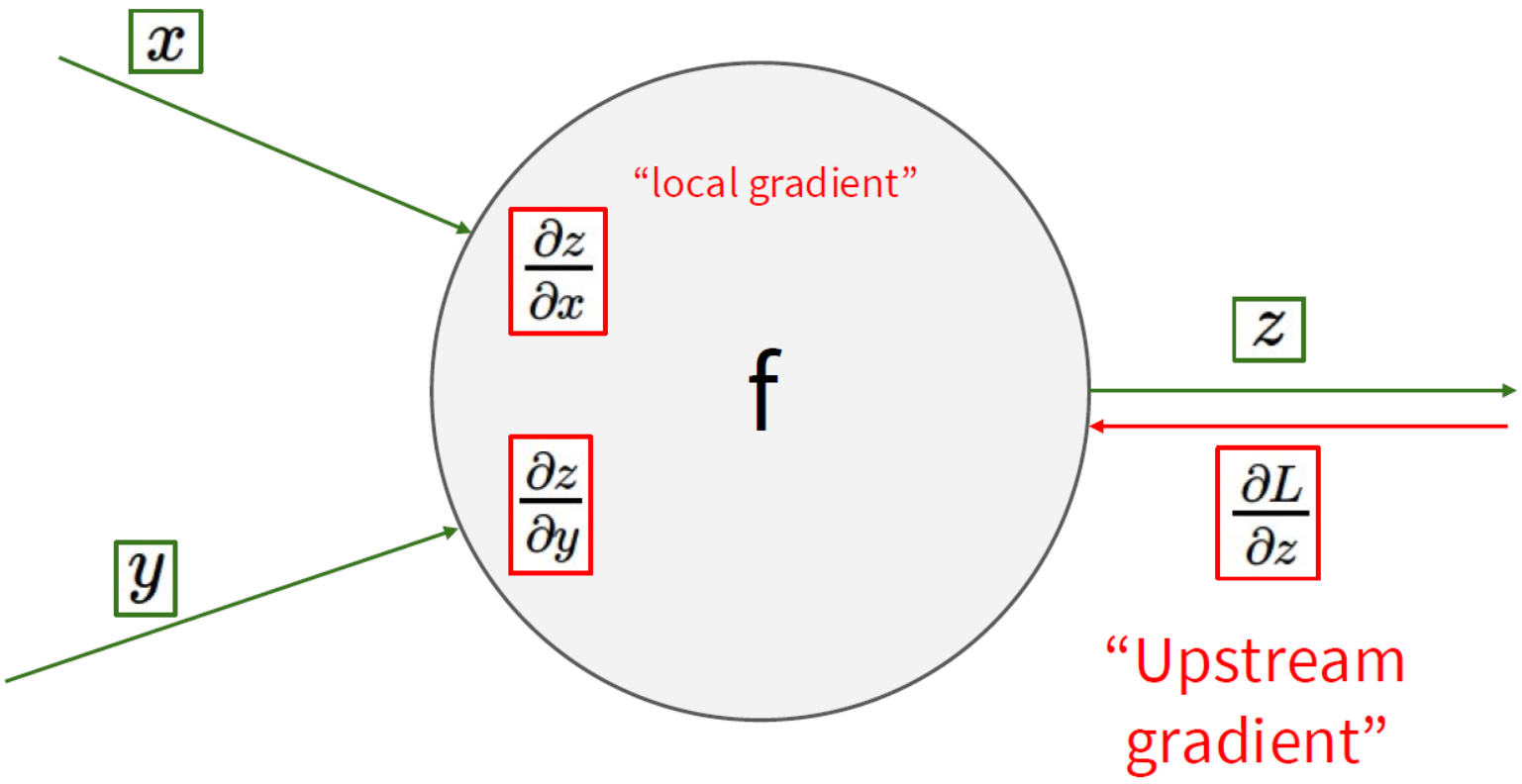
Back Propagation: (forward direction)



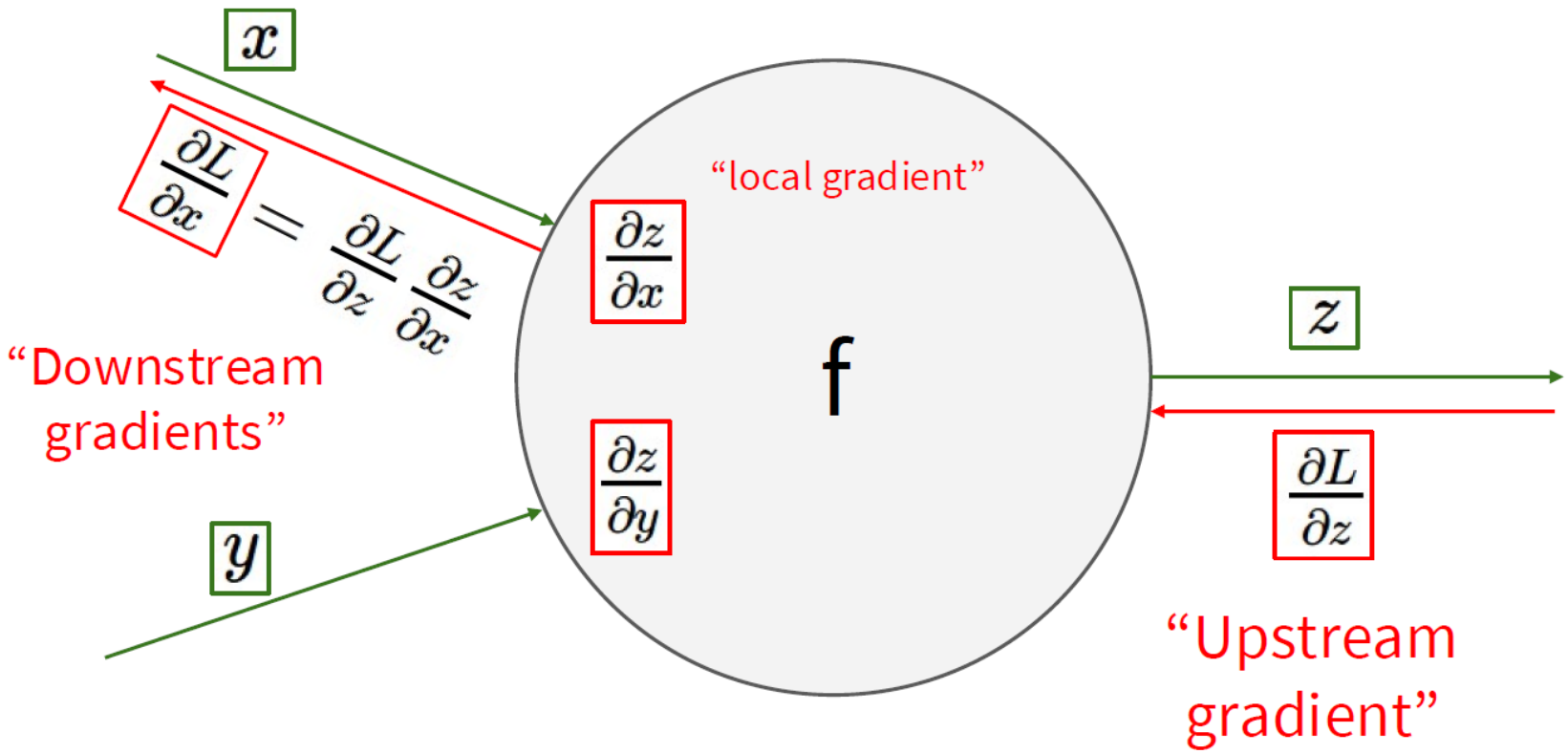
Back Propagation: (forward direction) compute local gradient



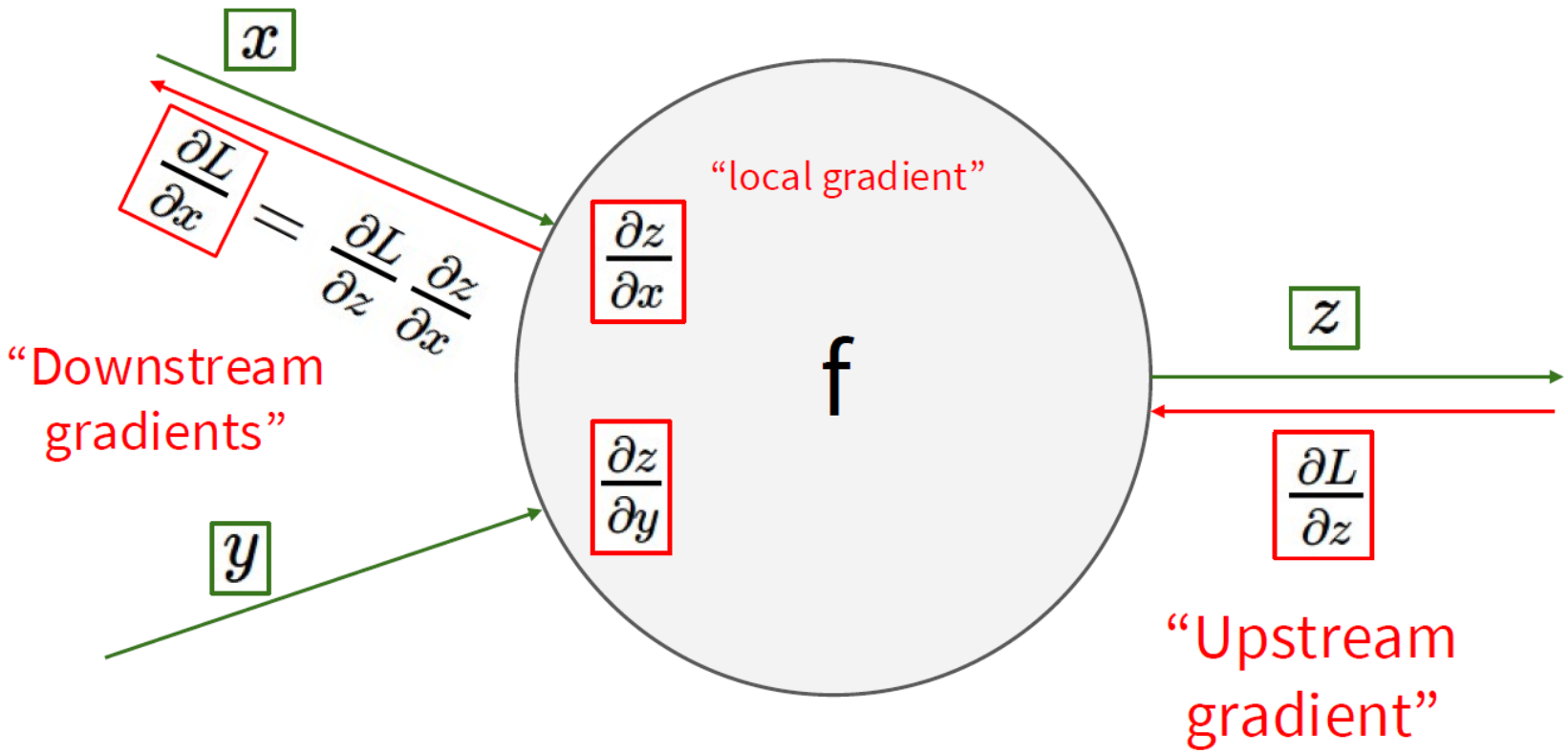
Back Propagation: (backward direction) receive upstream gradient (error signal)



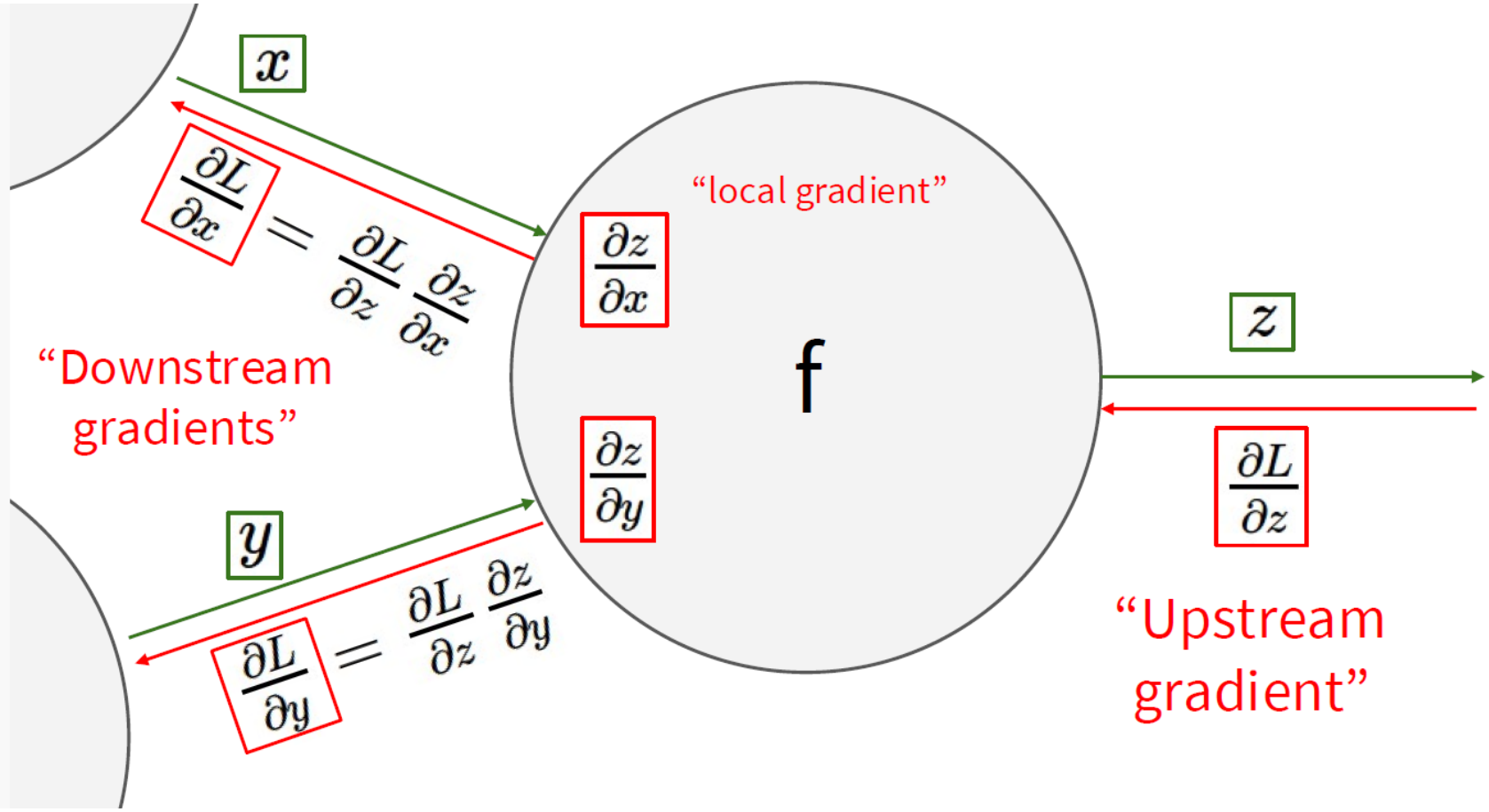
Back Propagation: (backward direction) compute downstream gradient



Back Propagation: (backward direction) compute downstream gradient



Back Propagation: propagate downstream gradient as upstream for the lower layer



Autograd framework

```
class Multiply(torch.autograd.Function):
  @staticmethod
  def forward(ctx, x, y):
    ctx.save_for_backward(x, y)
    z = x * y
    return z
  @staticmethod
  def backward(ctx, grad_z):
    x, y = ctx.saved_tensors
    grad_x = y * grad_z   # dz/dx * dL/dz
    grad_y = x * grad_z   # dz/dy * dL/dz
    return grad_x, grad_y
```

Need to cache some values for use in backward

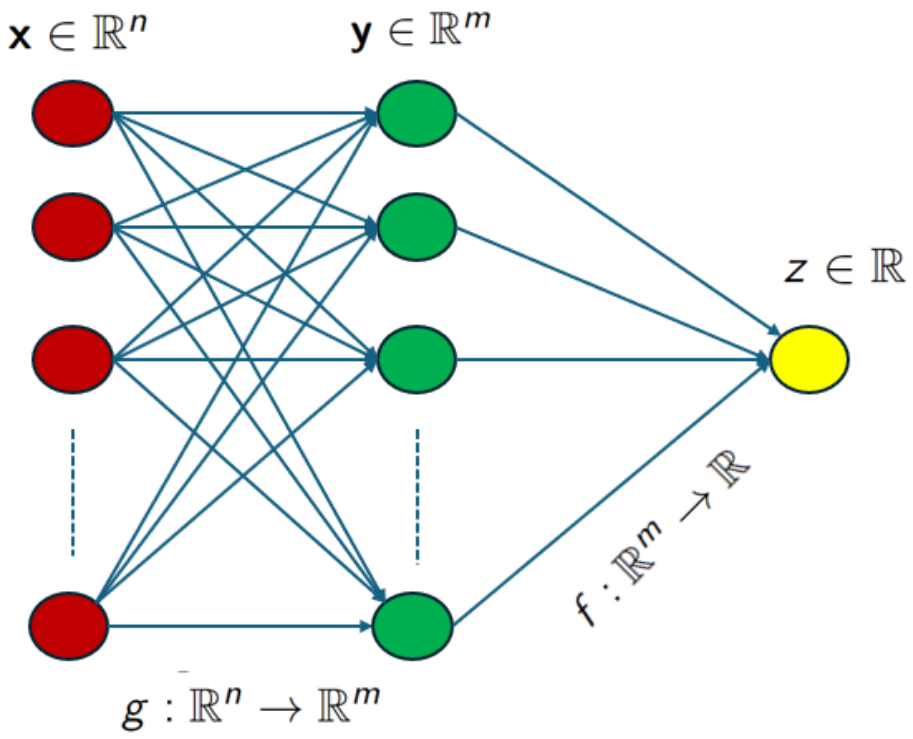
Upstream gradient

Multiply upstream and local gradients

Single train cycle. in 5 main steps

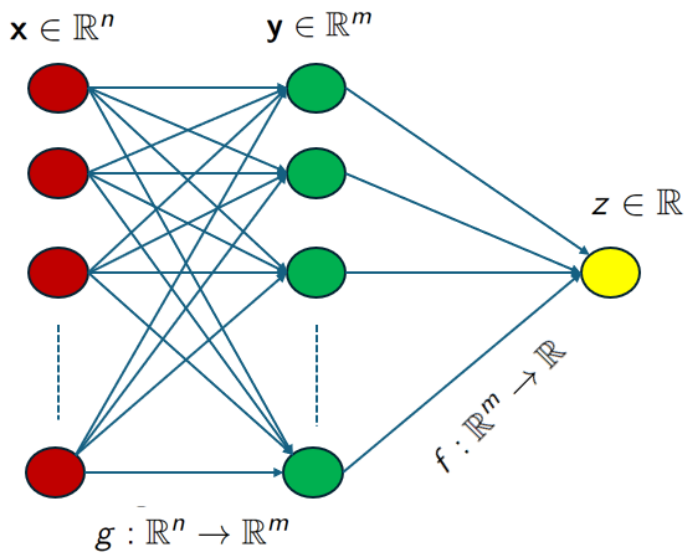
```
optimizer.zero_grad()   # (1) Clear old gradients

output = model(input)   # (2) Forward pass (build graph)

loss = criterion(output, target)

loss.backward()         # (3) Backward pass (compute gradients)

optimizer.step()        # (4) Update parameters
```

- **High-dimensional chain rule:** given $z \in \mathbb{R}$, $\mathbf{y} \in \mathbb{R}^m$, $\mathbf{x} \in \mathbb{R}^n$, $g : \mathbb{R}^n \to \mathbb{R}^m$,$f : \mathbb{R}^m \to \mathbb{R}$



- **High-dimensional chain rule:** given $z \in \mathbb{R}$, $\mathbf{y} \in \mathbb{R}^m$, $\mathbf{x} \in \mathbb{R}^n$, $g : \mathbb{R}^n \to \mathbb{R}^m$,$f : \mathbb{R}^m \to \mathbb{R}$

$$\frac{d}{dx_i} f(g(\mathbf{x})) = \sum_{j=1}^{m} \frac{\partial y_j}{\partial x_i} \frac{\partial z}{\partial y_j} = \frac{\partial \mathbf{y}}{\partial x_i} \frac{\partial z}{\partial \mathbf{y}} \tag{43}$$



- **High-dimensional chain rule:** given $z \in \mathbb{R}$, $\mathbf{y} \in \mathbb{R}^m$, $\mathbf{x} \in \mathbb{R}^n$, $g : \mathbb{R}^n \to \mathbb{R}^m$,$f : \mathbb{R}^m \to \mathbb{R}$

$$\frac{d}{dx_i} f(g(\mathbf{x})) = \sum_{j=1}^{m} \frac{\partial y_j}{\partial x_i} \frac{\partial z}{\partial y_j} = \frac{\partial \mathbf{y}}{\partial x_i} \frac{\partial z}{\partial \mathbf{y}} \tag{44}$$

- Jacobian Matrix $J_g(\mathbf{x})$:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} & \cdots & \frac{\partial y_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \frac{\partial y_m}{\partial x_2} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix} \in \mathbb{R}^{m \times n} \tag{45}$$

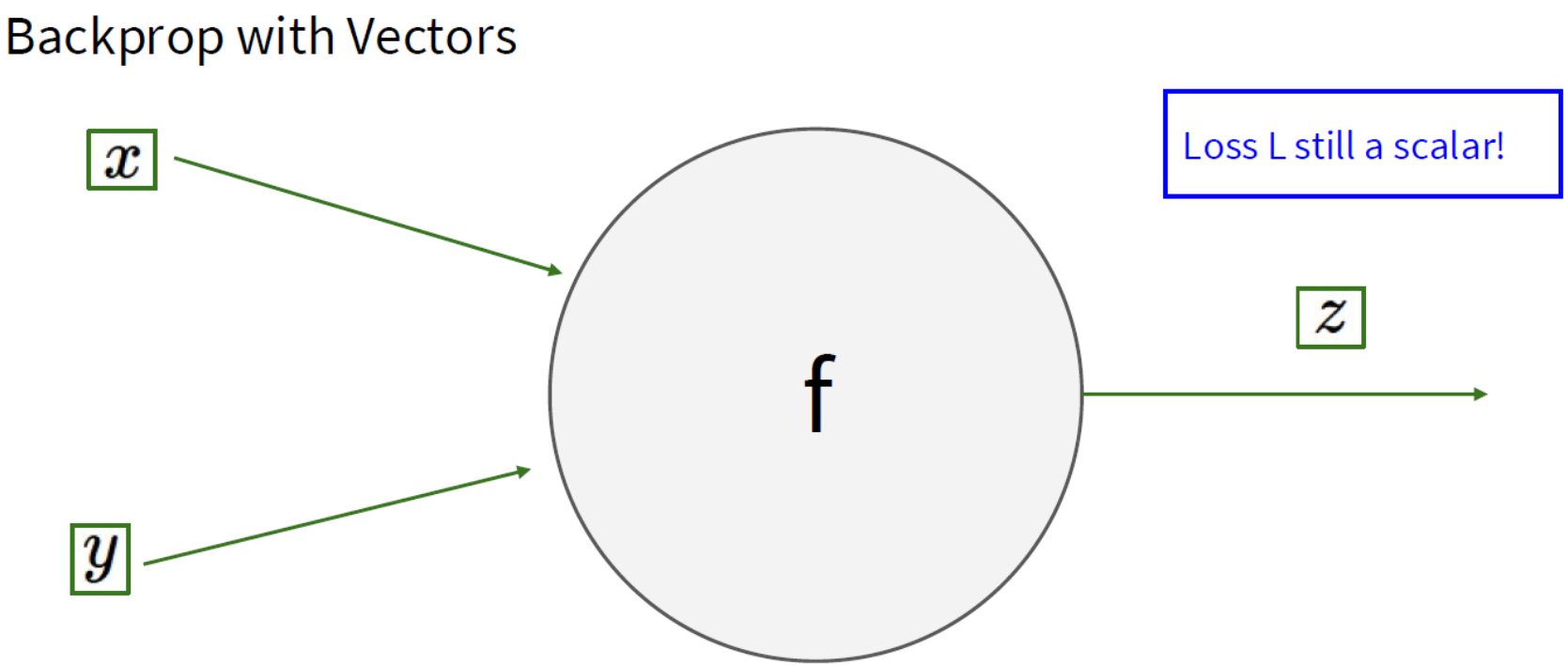
- $\frac{d}{dx_i} f(g(\mathbf{x})) =$

$$(\frac{\partial \mathbf{y}}{\partial \mathbf{x}}^T \frac{\partial z}{\partial \mathbf{y}})[i] \tag{46}$$

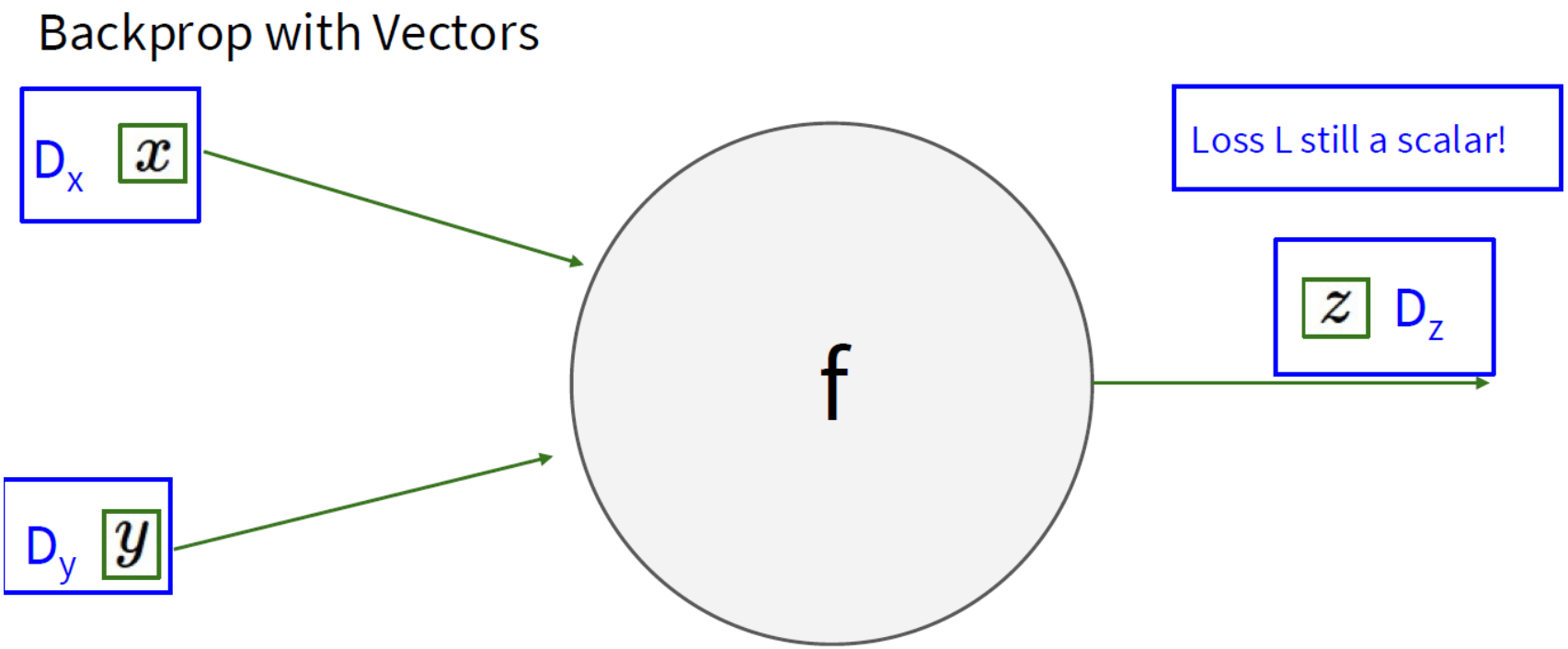
- Example: Let $\mathbf{y} = \mathbf{W}\mathbf{x}$ and a scalar loss defined over y, $\mathcal{L} = f(y)$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{W}^T \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \tag{47}$$

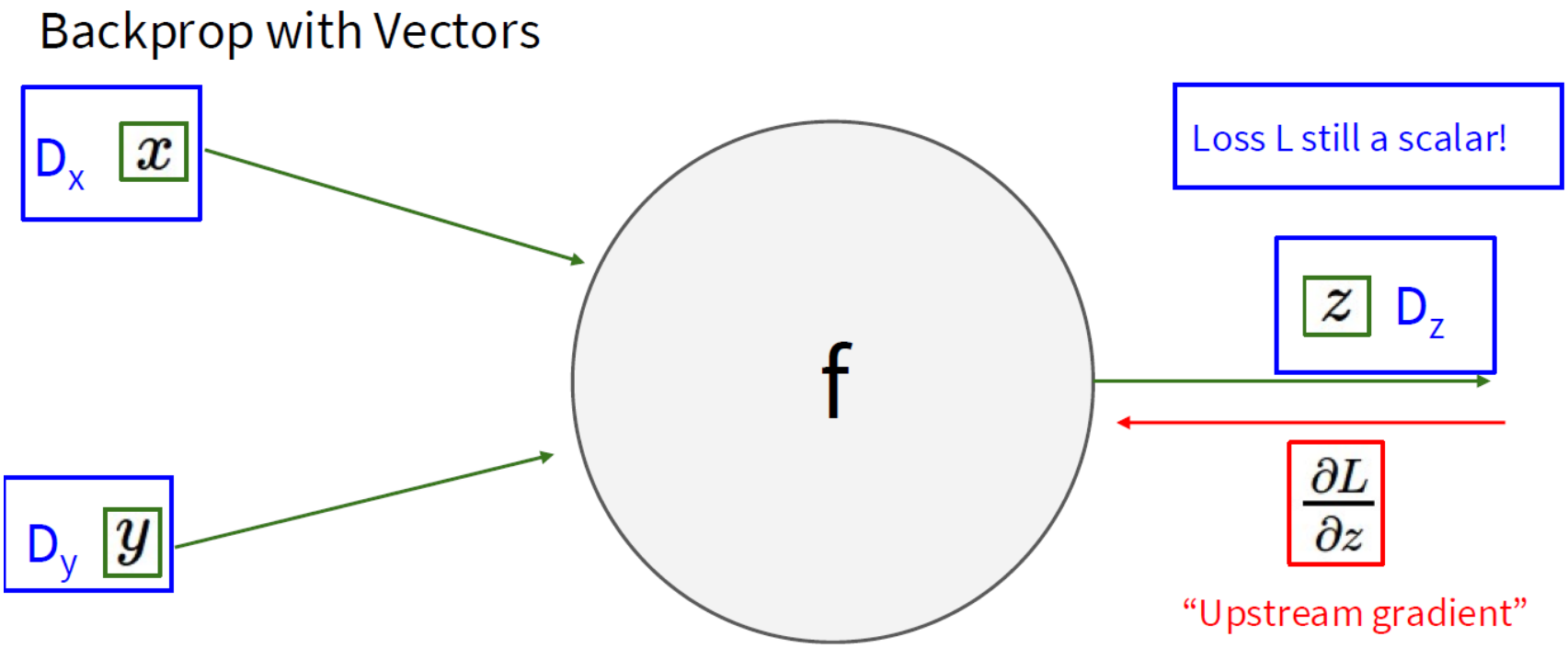
Back Propagation: (backward direction) compute downstream gradient



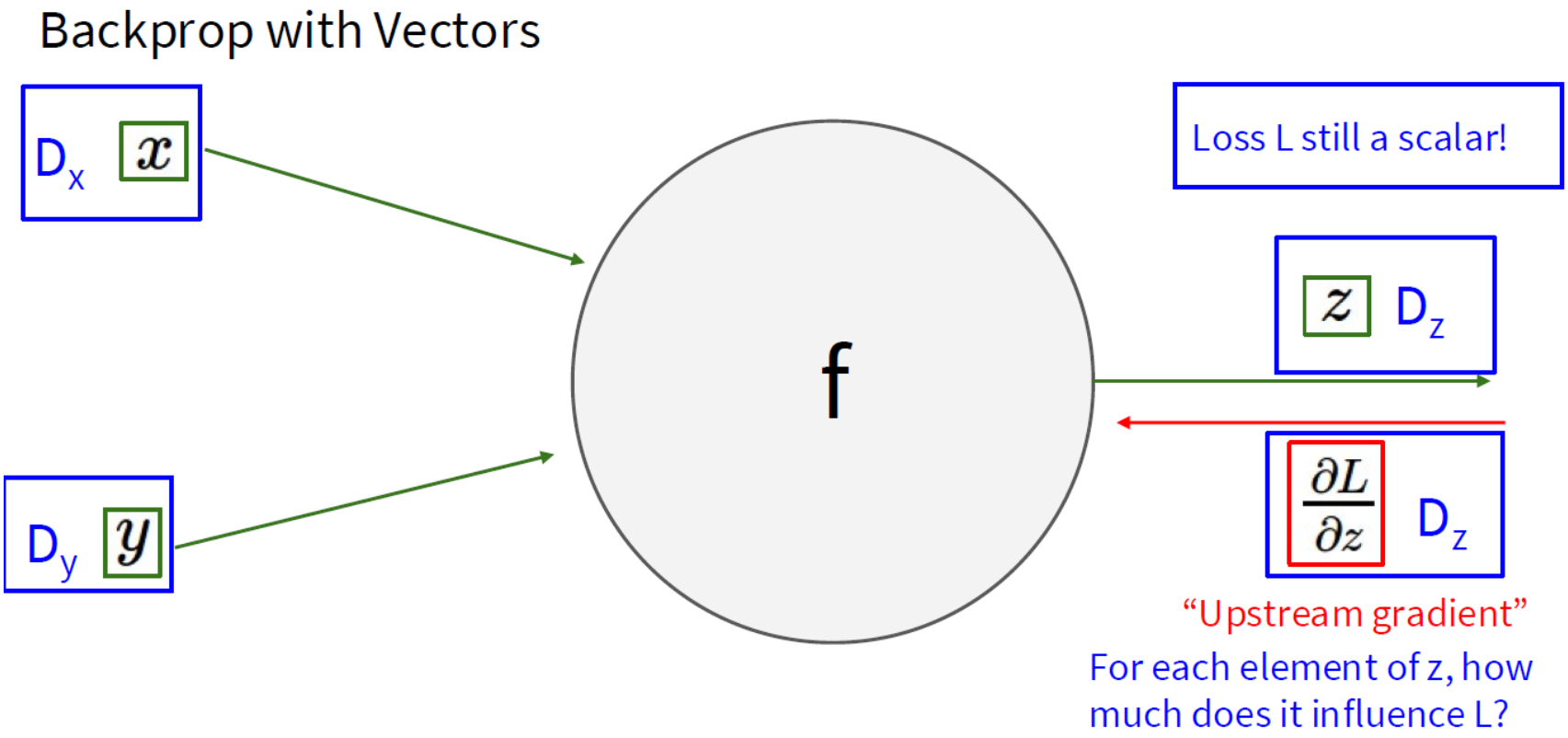
Back Propagation: (backward direction) compute downstream gradient



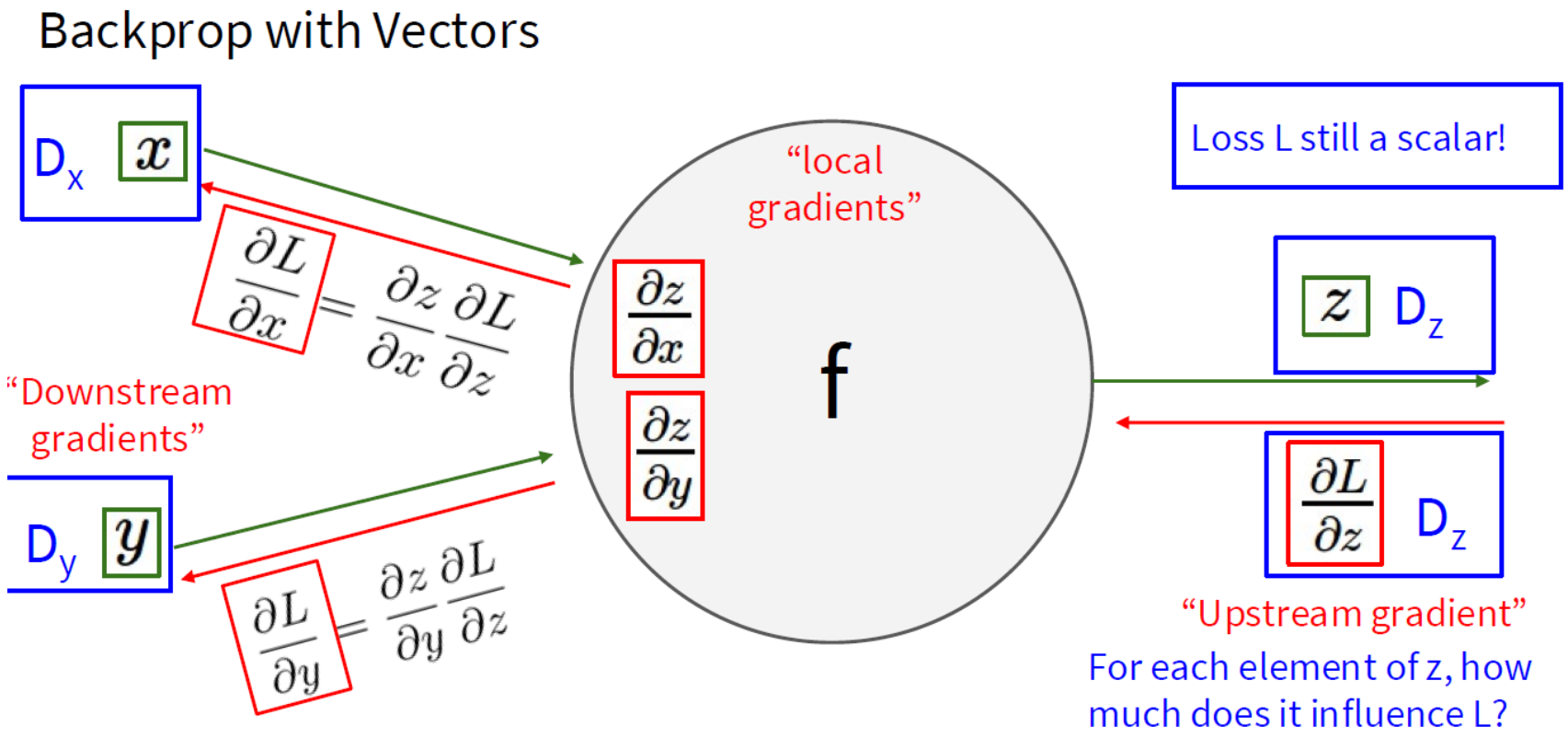
Back Propagation: (backward direction) compute downstream gradient



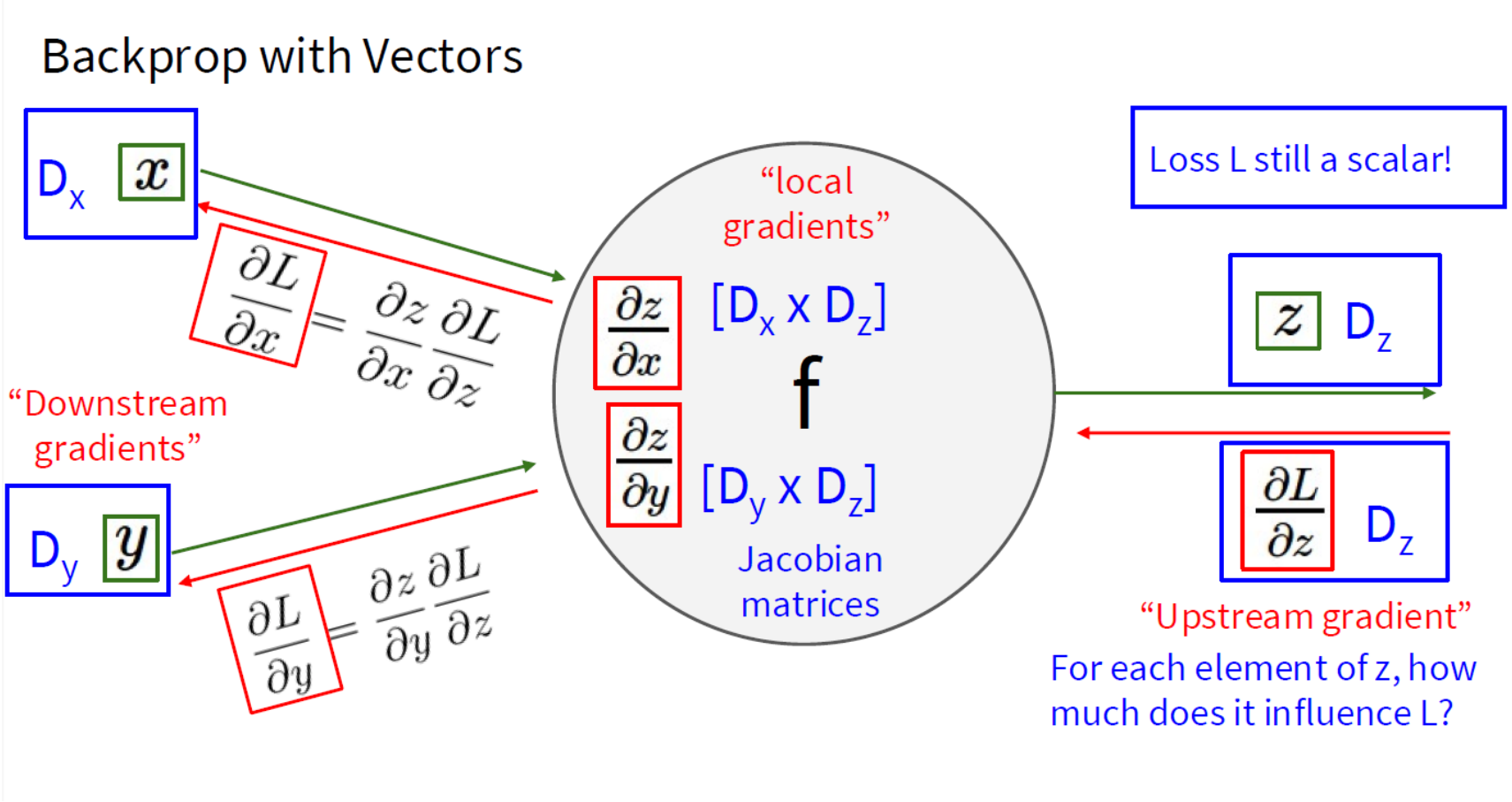
Back Propagation: (backward direction) compute downstream gradient



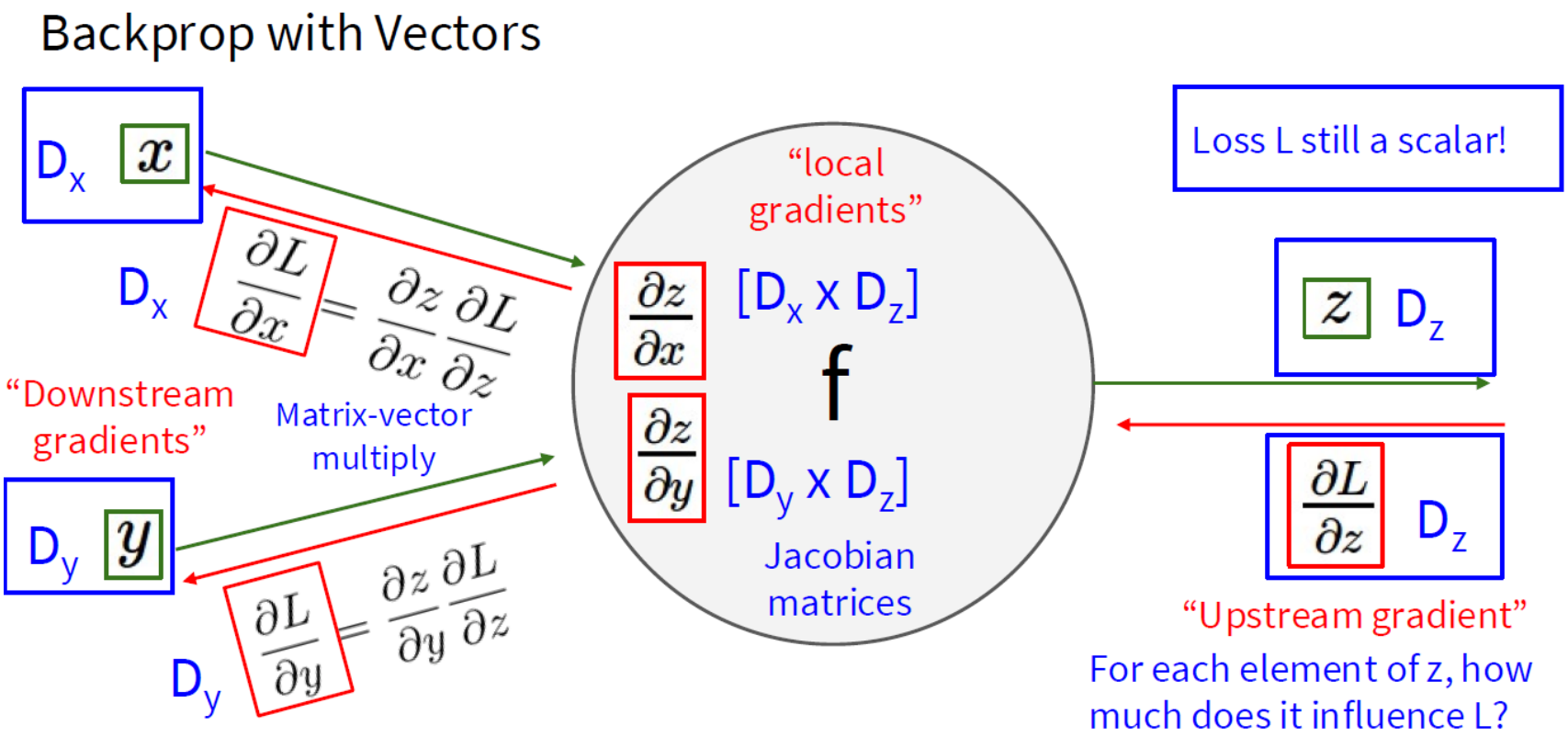
Back Propagation: (backward direction) compute downstream gradient



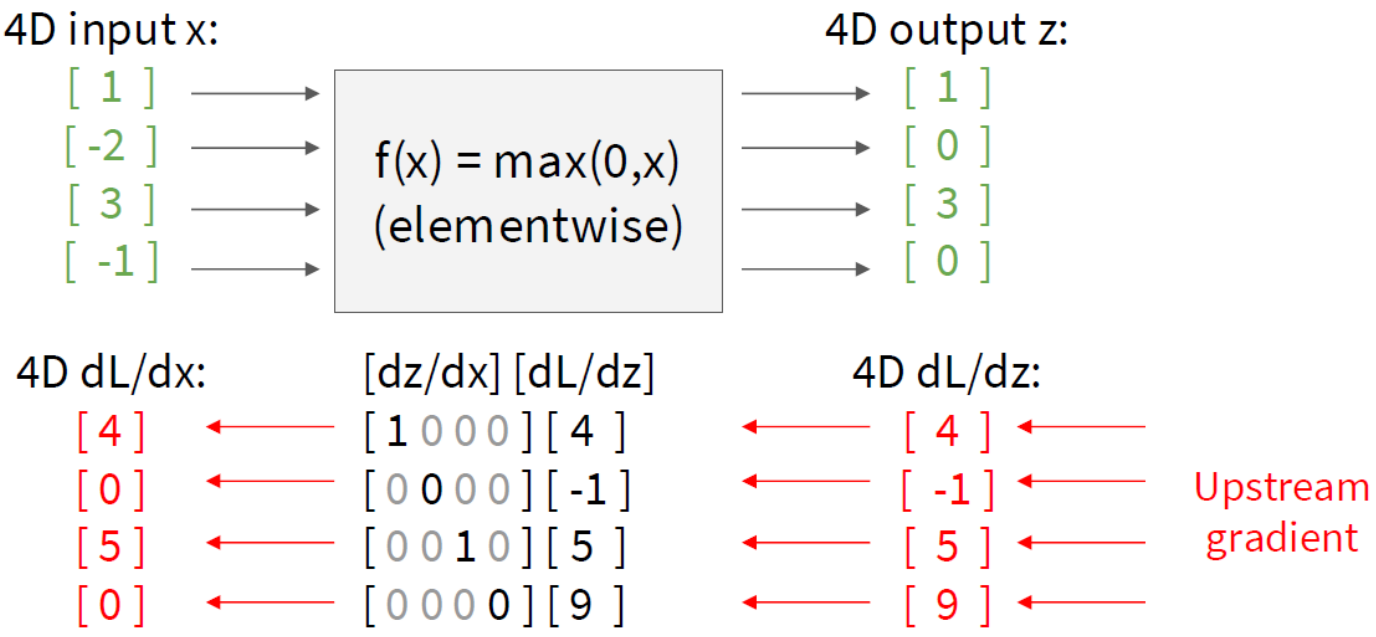
Back Propagation: (backward direction) compute downstream gradient



Back Propagation: (backward direction) compute downstream gradient



Back Propagation: *max*() operator



Sharpness-aware minimization

SAM reformulates training as a min–max optimization problem:

$$\min_{w} \max_{\|\epsilon\| \leq \rho} \mathcal{L}(w + \epsilon), \tag{48}$$

where ϵ denotes a perturbation in parameter space and $\rho > 0$ controls the size of the neighborhood.

- The inner maximization is approximated using a first-order Taylor expansion:

$$\mathcal{L}(w + \epsilon) \approx \mathcal{L}(w) + \epsilon^{\top} \nabla_w \mathcal{L}(w). \tag{49}$$

  Maximizing the linearized loss under the constraint $\|\epsilon\| \leq \rho$ yields the optimal perturbation:

$$\epsilon^* = \rho \frac{\nabla_w \mathcal{L}(w)}{\|\nabla_w \mathcal{L}(w)\|}. \tag{50}$$

- Using ϵ^*, the SAM update becomes:

$$w_{t+1} = w_t - \eta \nabla_w \mathcal{L}(w_t + \epsilon^*), \tag{51}$$

  where η is the learning rate.

Sharpness-aware minimization

SAM minimizes the worst-case loss in a local neighborhood of the parameters. This encourages solutions with low curvature (flat minima), which are empirically and theoretically associated with improved generalization.

Lemma (Convexity Lemma)

Let $f : \mathbb{R}^d \to \mathbb{R}$ be convex and differentiable. Then, for all $x, y \in \mathbb{R}^d$,

$$f(x) \geq f(y) + \langle \nabla f(y), x - y \rangle. \tag{52}$$

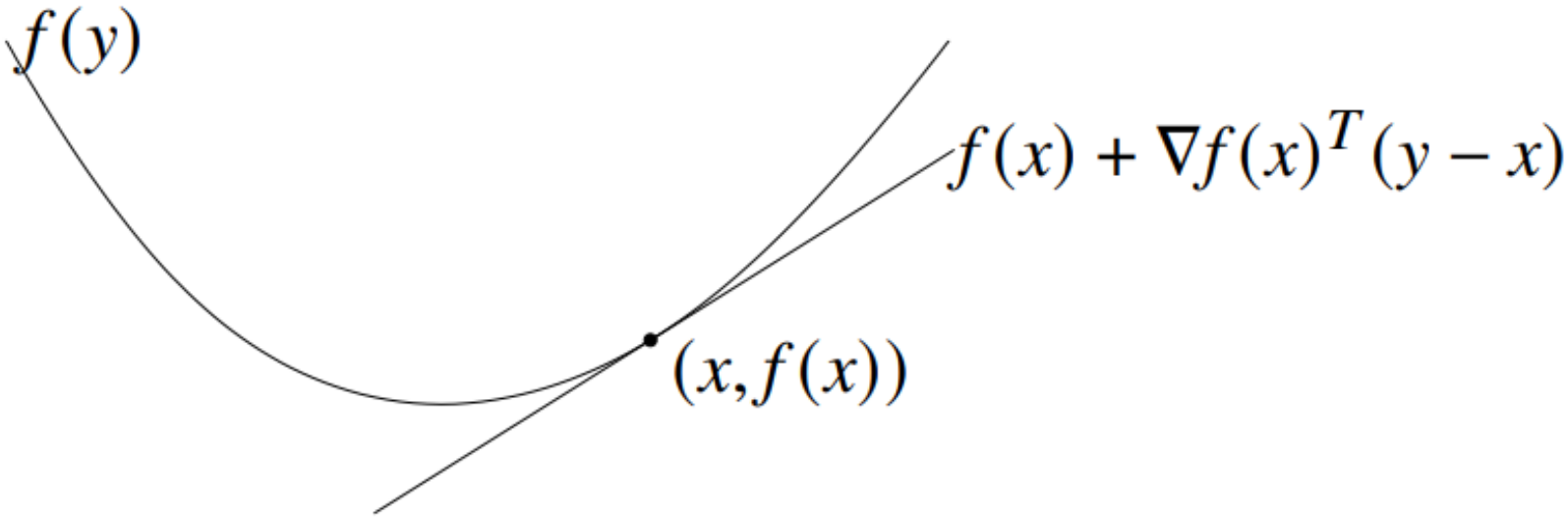


Figure 17: Convex function

Gradient descent

Proof.

By convexity of f, for all $t \in (0,1)$ we have

$$f\big(y + t(x-y)\big) \leq tf(x) + (1-t)f(y). \tag{53}$$

Subtracting $f(y)$ from both sides and dividing by t yields

$$\frac{f\big(y + t(x-y)\big) - f(y)}{t} \leq f(x) - f(y). \tag{54}$$

Taking the limit as $t \to 0$ and using differentiability of f at y, we obtain

$$\langle \nabla f(y), x - y \rangle \leq f(x) - f(y), \tag{55}$$

which proves (52). $\square$

Gradient descent

Definition (Jacobian)

Let $F : \mathbb{R}^d \to \mathbb{R}^p$ be differentiable, and let $x \in \mathbb{R}^d$. We denote by $DF(x)$ the *Jacobian* of F at x, which is the matrix of first-order partial derivatives defined as

$$[DF(x)]_{ij} = \frac{\partial f_i}{\partial x_j}(x), \qquad i = 1, \ldots, p, \;\; j = 1, \ldots, d, \tag{56}$$

where we write

$$F(x) = \big(f_1(x), \ldots, f_p(x)\big).$$

Consequently, the Jacobian matrix satisfies $DF(x) \in \mathbb{R}^{p \times d}$.

Gradient descent

Definition (Lipschitz Continuity)

Let $F : \mathbb{R}^d \to \mathbb{R}^p$ and let $L > 0$. We say that F is *L-Lipschitz* if for all $x, y \in \mathbb{R}^d$,

$$\|F(y) - F(x)\| \leq L\|y - x\|. \tag{57}$$

Lemma

Let $F : \mathbb{R}^d \to \mathbb{R}^p$ be differentiable and let $L > 0$. Then F is L-Lipschitz if and only if

$$\|DF(x)\| \leq L, \quad \forall x \in \mathbb{R}^d. \tag{58}$$

Gradient descent

Proof.

($\Rightarrow$) Assume that F is L-Lipschitz. For any $x \in \mathbb{R}^d$ and any direction $h \in \mathbb{R}^d$ with $\|h\| = 1$, we have

$$\frac{\|F(x+th) - F(x)\|}{t} \leq L \tag{59}$$

for all $t > 0$. Taking the limit as $t \to 0$ and using differentiability of F yields

$$\|DF(x)h\| \leq L. \tag{60}$$

Since this holds for all unit vectors h, it follows that

$$\|DF(x)\| \leq L. \tag{61}$$

$\square$

Proof.

($\Leftarrow$) Conversely, assume that $\|DF(x)\| \leq L$ for all $x \in \mathbb{R}^d$. For any $x, y \in \mathbb{R}^d$, define the function

$$\gamma(t) = F\big(x + t(y - x)\big), \quad t \in [0, 1].$$

By the fundamental theorem of calculus,

$$F(y) - F(x) = \int_0^1 DF\big(x + t(y - x)\big)(y - x)\, dt. \tag{62}$$

Taking norms and using the operator norm bound,

$$\|F(y) - F(x)\| \leq \int_0^1 \|DF(x + t(y - x))\|\, \|y - x\|\, dt \tag{63}$$

$$\leq L\|y - x\|. \tag{64}$$

Thus, F is L-Lipschitz. $\square$

Theorem (Fundamental Theorem of Calculus in $\mathbb{R}^d$)

Let $F : \mathbb{R}^d \to \mathbb{R}^p$ be differentiable. Then for any $x, y \in \mathbb{R}^d$,

$$F(y) - F(x) = \int_0^1 DF\big(x + t(y - x)\big)\,(y - x)\,dt, \tag{65}$$

where $DF(\cdot)$ denotes the Jacobian matrix of F.

Proof.

Fix $x, y \in \mathbb{R}^d$ and define the curve

$$\gamma(t) = x + t(y - x), \quad t \in [0, 1]. \tag{66}$$

Define the function $G : [0, 1] \to \mathbb{R}^p$ by

$$G(t) = F(\gamma(t)) = F(x + t(y - x)). \tag{67}$$

Since F is differentiable and γ is differentiable, the chain rule implies that G is differentiable, with derivative

$$G'(t) = DF\big(x + t(y - x)\big)\,(y - x). \tag{68}$$

$\square$

Proof.

Applying the one-dimensional fundamental theorem of calculus to G, we obtain

$$G(1) - G(0) = \int_0^1 G'(t)\, dt. \tag{69}$$

Substituting the expression for $G'(t)$ yields

$$F(y) - F(x) = \int_0^1 DF\big(x + t(y - x)\big)\,(y - x)\, dt, \tag{70}$$

which completes the proof. $\square$

Proof of Descent Lemma

Proof.

Fix $x, y \in \mathbb{R}^d$ and define the function

$$\varphi(t) := f\big(x + t(y - x)\big), \quad t \in [0, 1].$$

By the Fundamental Theorem of Calculus, we have

$$f(y) = f(x) + \int_0^1 \big\langle \nabla f\big(x + t(y - x)\big),\, y - x \big\rangle \, dt. \tag{71}$$

Adding and subtracting $\nabla f(x)$ inside the inner product yields

$$\begin{aligned} f(y) = f(x) &+ \langle \nabla f(x), y - x \rangle \\ &+ \int_0^1 \big\langle \nabla f\big(x + t(y - x)\big) - \nabla f(x),\, y - x \big\rangle \, dt. \end{aligned} \tag{72}$$

$\square$

Proof of Descent Lemma

Proof.

Applying the Cauchy–Schwarz inequality gives

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \int_0^1 \left\| \nabla f\big(x + t(y-x)\big) - \nabla f(x) \right\| \, \|y - x\| \, dt. \tag{73}$$

Since ∇f is L-Lipschitz, we have

$$\left\| \nabla f\big(x + t(y-x)\big) - \nabla f(x) \right\| \leq Lt\|y - x\|.$$

Substituting this bound into the integral yields

$$\begin{aligned} f(y) &\leq f(x) + \langle \nabla f(x), y - x \rangle + \int_0^1 Lt\|y - x\|^2 \, dt \\ &= f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2}\|y - x\|^2, \end{aligned} \tag{74}$$

which proves (**??**). □